

Algorytmy Zaawansowane

Rozwiązania egzaminów z lat ubiegłych

Rozwiązujemy zadania wymagające pracy; te będące powtórzeniem materiału z wykładu odsyłają do notatek.

Spis treści

1	Chwyty egzaminacyjne	2
1.1	Ta sama procedura, inna metryka	2
2	2018	3
2.1	Egzamin zerowy	3
2.2	Termin 1	4
2.3	Termin 2	5
2.4	Termin poprawkowy	6
3	2021	7
3.1	Termin 1	7
3.2	Termin 2	8
3.3	Termin 3	9
A	2018 — rozwiązania	10
A.1	Egzamin zerowy	10
A.2	Termin 1	16
A.3	Termin 2	22
A.4	Termin poprawkowy	27
B	2021 — rozwiązania	32
B.1	Termin 1	32
B.2	Termin 2	36
B.3	Termin 3	40
C	Zadania różne	44

Chwyty egzaminacyjne

Zadania egzaminacyjne rzadko wymagają nowego algorytmu od zera — najczęściej są wariantem czegoś z wykładu. Poniżej zbieramy wzorce, które wracają.

Ta sama procedura, inna metryka

Algorytm „dziel i zwyciężaj” dla pary najbliższych punktów nie zależy od tego, *jaką* metrykę d liczymy — zależy tylko od jednego faktu upakowania: w prostokącie o bokach $\mathcal{O}(\delta)$ mieści się $\mathcal{O}(1)$ punktów parami odległych o $\geq \delta$. Dopóki ten fakt zachodzi, faza łączenia kosztuje $\mathcal{O}(n)$, a całość $\mathcal{O}(n \log n)$ — niezależnie od metryki.

Stąd przepis: problem „czy istnieje bliska para” przekształć w parę najbliższych w odpowiedniej metryce, uruchom standardowy algorytm, porównaj wynik z progiem.

Metryka	Wzór	Gdzie
euklidesowa ℓ_2	$\sqrt{(\Delta x)^2 + (\Delta y)^2}$	para najbliższych z wykładu
Czebyszewa ℓ_∞	$\max(\Delta x , \Delta y)$	kwadraty jednostkowe (2021, T1, Zad. 1)
miejska ℓ_1	$ \Delta x + \Delta y $	obrót o 45° sprowadza do ℓ_∞

Dlaczego ℓ_∞ dla kwadratów. Dwa osiowe kwadraty o boku 1 przecinają się $\iff$ nakładają się ich rzuty na obie osie, czyli $|\Delta x| \leq 1$ oraz $|\Delta y| \leq 1$, czyli $\ell_\infty \leq 1$. Najmniejsza para w ℓ_∞ i porównanie z 1 rozstrzyga problem.

Dlaczego ℓ_1 sprowadza się do ℓ_∞ . Odwzorowanie $(x, y) \mapsto (x+y, x-y)$ przeprowadza ℓ_1 w ℓ_∞ (z czynnikiem stałym), więc wystarczy ta sama procedura na obróconych współrzędnych. Stała w fakcie upakowania zmienia się (inny kształt „kuli”), ale pozostaje $\mathcal{O}(1)$ — złożoność bez zmian.

2018

Egzamin zerowy

Zadanie 1 — dwa najtańsze symbole pod wspólnym rodzicem (15 p.)

Udowodnij, że w kodowaniu Huffmana istnieje optymalne drzewo kodowe, w którym dwa najtańsze (najrzadziej występujące) symbole mają wspólnego rodzica.

Zadanie 2 — skojarzenie najliczniejsze a ścieżki powiększające (15 p.)

Udowodnij, że skojarzenie w grafie jest najliczniejsze wtedy i tylko wtedy, gdy nie istnieje ścieżka powiększająca.

Zadanie 3 — matroid jednego wierzchołka na składową (15 p.)

Niech G będzie grafem, a $\mathcal{A}_G$ rodziną podzbiorów $V(G)$ taką, że $S \in \mathcal{A}_G$ wtedy i tylko wtedy, gdy każda składowa spójności G zawiera co najwyżej jeden wierzchołek z S . Rozstrzygnij, czy para $(V(G), \mathcal{A}_G)$ zawsze jest matroidem (udowodnij to lub podaj kontrprzykład).

Zadanie 4 — dokładne pokrycie wierzchołkowe w czasie podwykładniczym (15 p.)

Zaprojektuj algorytm dokładny, który znajdzie licznosc najmniejszego pokrycia wierzchołkowego grafu w czasie $o(2^n)$, gdzie n jest liczbą wierzchołków. Uzasadnij poprawność i oszacuj złożoność swojego algorytmu.

Zadanie 5 — 3-aproksymacja problemu 3DC (15 p.)

Rozważmy problem — nazwijmy go roboczo 3DC — w którym mamy daną rodzinę $\mathcal{R}$ 3-elementowych podzbiorów pewnego zbioru X i poszukujemy jak najmniej licznego zbioru $S \subseteq X$ takiego, że każdy zbiór z $\mathcal{R}$ zawiera przynajmniej jeden element z S . Zaprojektuj wielomianowy algorytm 3-aproksymacyjny dla problemu 3DC (i udowodnij, że stała aproksymacji istotnie jest nie większa niż 3). Możesz zainspirować się algorytmem 2-aproksymacyjnym dla problemu najmniejszego pokrycia wierzchołkowego.

Termin 1

Zadanie 1 — wszystkie liście Huffmana na tej samej głębokości (15 p.)

Niech T będzie optymalnym drzewem kodowym w kodowaniu Huffmana dla alfabetu A oraz funkcji częstości $f : A \rightarrow \mathbb{N}$. Przypuśćmy, że $|A| = 2^k$ oraz dla każdych trzech symboli $x, y, z \in A$ zachodzi $f(x) + f(y) > f(z)$. Udowodnij, że w drzewie T wszystkie liście znajdują się na tej samej głębokości (równej k).

Zadanie 2 — najtańszy element w bazie minimalnej (15 p.)

Rozważmy matroid $M = (S, \mathcal{I})$ oraz funkcję wag $w : S \rightarrow \mathbb{R}_+$. Niech x będzie elementem S minimalizującym $w(x)$. Udowodnij, że x należy do pewnej bazy w M o minimalnej wadze (innymi słowy: istnieje zbiór $I \in \mathcal{I}$ taki, że $x \in I$ oraz $w(I) = \min_{J \in \mathcal{I}} w(J)$).

Zadanie 3 — mnożenie macierzy z 32 mnożeniami bloków (15 p.)

Przypuśćmy, że istnieją wzory pozwalające pomnożyć dwie macierze 4×4 wykonując 256 elementarnych operacji dodawania liczb i 32 elementarne operacje mnożenia liczb (wzory nie korzystają z przemienności mnożenia). Skonstruuj na tej podstawie wydajny algorytm mnożenia macierzy; wyznacz złożoność tego algorytmu.

Zadanie 4 — minimalny podział grafu na trójkąty (15 p.)

Przez podział grafu G na trójkąty rozumiemy podział zbioru $V(G)$ na trzelementowe podzbiory $\{u_i, v_i, w_i\}$ ($i = 1, \dots, k$), gdzie dla każdego i zachodzi $u_i v_i, v_i w_i, u_i w_i \in E(G)$. Dla wagi $w : E(G) \rightarrow \mathbb{R}$ przez wagę podziału rozumiemy $\sum_{i=1}^k (w(u_i v_i) + w(v_i w_i) + w(u_i w_i))$. W problemie MINIMUM TRIANGLE PARTITION mamy dany graf ważony krawędziowo i poszukujemy minimalnej wagi podziału na trójkąty. Zaprojektuj algorytm rozwiązujący ten problem w czasie $\mathcal{O}^*(2^n)$, gdzie $n = |V(G)|$; uzasadnij poprawność i oszacuj czas działania. *Wskazówka:* programowanie dynamiczne.

Zadanie 5 — 3-aproksymacyjne kolorowanie grafów dysków jednostkowych (15 p.)

Podaj wielomianowy algorytm 3-aproksymacyjny dla problemu kolorowania grafów przecięć dysków jednostkowych i udowodnij, że współczynnik aproksymacji istotnie wynosi 3. Możesz pominąć dowód potrzebnego geometrycznego faktu (ale musisz go sformułować).

Termin 2

Zadanie 1 — ścieżki powiększające zbioru niezależnego w grafie bezpazurowym (15 p.)

Graf G jest *bezpazurowy*, jeżeli żaden wierzchołek G nie ma trzech niezależnych sąsiadów. Niech I będzie zbiorem niezależnym w grafie. Ścieżkę $P = v_1 v_2 \dots v_m$ nazywamy *powiększającą* dla I , jeżeli P ma nieparzystą liczbę wierzchołków, wierzchołki o parzystych indeksach należą do I , wierzchołki o nieparzystych indeksach nie należą do I oraz G nie zawiera żadnej krawędzi między wierzchołkami niesąsiadującymi na ścieżce. Niech G będzie grafem bezpazurowym, a I zbiorem niezależnym w G . Udowodnij, że jeżeli I nie jest najliczniejszym zbiorem niezależnym w G , to istnieje ścieżka powiększająca dla I . *Wskazówka:* zainspiruj się dowodem analogicznego stwierdzenia dla skojarzeń.

Zadanie 2 — matroid z ograniczeniami licznosciowymi na sumie zbiorów (15 p.)

Niech A i B będą niepustymi zbiorami rozłącznymi, a k dodatnią liczbą naturalną. Niech $\mathcal{F}$ będzie rodziną podzbiorów zbioru $A \cup B$ taką, że $S \in \mathcal{F}$ wtedy i tylko wtedy, gdy $|S \cap B| \leq k$ oraz $|S \cap A| \leq |S \cap B|$. Rozstrzygnij, czy para $(A \cup B, \mathcal{F})$ zawsze jest matroidem (udowodnij to lub podaj kontrprzykład).

Zadanie 3 — para najbliższych punktów w czasie $n \log n$ (15 p.)

Podaj algorytm, który znajduje parę najbliższych punktów na płaszczyźnie w czasie $\mathcal{O}(n \log n)$, gdzie n jest liczbą punktów. Uzasadnij jego poprawność i oszacuj czas działania. Możesz pominąć dowód potrzebnego geometrycznego faktu (ale musisz go sformułować).

Zadanie 4 — minimalny podział grafu na trójkąty (15 p.)

Przez podział grafu G na trójkąty rozumiemy podział zbioru $V(G)$ na trzejelementowe podzbiory $\{u_i, v_i, w_i\}$ ($i = 1, \dots, k$), gdzie dla każdego i zachodzi $u_i v_i, v_i w_i, u_i w_i \in E(G)$. Dla wagi $w : E(G) \rightarrow \mathbb{R}$ przez wagę podziału rozumiemy $\sum_{i=1}^k (w(u_i v_i) + w(v_i w_i) + w(u_i w_i))$. W problemie MINIMUM TRIANGLE PARTITION poszukujemy minimalnej wagi podziału danego grafu ważonego na trójkąty. Zaprojektuj algorytm rozwiązujący ten problem w czasie $\mathcal{O}^*(2^n)$, gdzie $n = |V(G)|$; uzasadnij poprawność i oszacuj czas działania. *Wskazówka:* programowanie dynamiczne.

Zadanie 5 — 3-aproksymacyjne kolorowanie grafów dysków jednostkowych (15 p.)

Podaj wielomianowy algorytm 3-aproksymacyjny dla problemu kolorowania grafów przecięć dysków jednostkowych i udowodnij, że współczynnik aproksymacji istotnie wynosi 3. Możesz pominąć dowód potrzebnego geometrycznego faktu (ale musisz go sformułować).

Termin poprawkowy**Zadanie 1 — niemalejące długości najkrótszych ścieżek powiększających (15 p.)**

Niech M będzie skojarzeniem w pewnym grafie G oraz niech P będzie najkrótszą ścieżką powiększającą dla M . Oznaczamy $M' := M \div P$. Niech P' będzie najkrótszą ścieżką powiększającą dla skojarzenia M' . Udowodnij, że $|P| \leq |P'|$ oraz że jeżeli P i P' mają wspólny wierzchołek, to $|P| < |P'|$.

Zadanie 2 — matroid podzbiorów krawędzi o ograniczonym stopniu (15 p.)

Niech $G = (V, E)$ będzie spójnym grafem dwudzielnym, a k dodatnią liczbą naturalną. Niech $\mathcal{F}$ będzie rodziną podzbiorów zbioru E taką, że $S \in \mathcal{F}$ wtedy i tylko wtedy, gdy każdy wierzchołek grafu G jest incydentny z co najwyżej k krawędziami z S . Rozstrzygnij, czy para $(E, \mathcal{F})$ zawsze jest matroidem (udowodnij to lub podaj kontrprzykład).

Zadanie 3 — mnożenie dużych liczb metodą Tooma-Cooka (15 p.)

Wiemy, że

$$(x_2 10^{2k} + x_1 10^k + x_0)(y_2 10^{2k} + y_1 10^k + y_0) = x_2 y_2 10^{4k} + \frac{C - 3A - B - 12x_2 y_2 + 3x_0 y_0}{6} 10^{3k} \\ + \left(\frac{A+B}{2} - x_2 y_2 - x_0 y_0 \right) 10^{2k} + \frac{6A - 2B - C + 12x_2 y_2 - 3x_0 y_0}{6} 10^k + x_0 y_0,$$

gdzie $A = (x_2 + x_1 + x_0)(y_2 + y_1 + y_0)$, $B = (x_2 - x_1 + x_0)(y_2 - y_1 + y_0)$ oraz $C = (4x_2 + 2x_1 + x_0)(4y_2 + 2y_1 + y_0)$. Wykorzystując te wzory, zaprojektuj wydajny algorytm mnożenia dużych liczb i oszacuj jego czas działania. *Wskazówka:* zainspiruj się algorytmem Karacuby.

Zadanie 4 — najkrótszy cykl Hamiltona (15 p.)

Podaj algorytm, który znajduje długość najkrótszego cyklu Hamiltona w grafie ważonym o n wierzchołkach w czasie $\mathcal{O}(2^n)$ (z dokładnością do czynnika wielomianowego). Udowodnij jego poprawność i oszacuj czas działania.

Zadanie 5 — 3-aproksymacyjny zbiór niezależny w grafach dysków jednostkowych (15 p.)

Zaprojektuj wielomianowy algorytm 3-aproksymacyjny dla problemu najliczniejszego zbioru niezależnego w grafach przecięć dysków jednostkowych i udowodnij, że współczynnik aproksymacji istotnie wynosi 3. Za rozwiązanie o gorszym współczynniku można dostać część punktów. *Wskazówka:* zainspiruj się algorytmem aproksymacyjnym dla liczby chromatycznej tych grafów.

2021

Termin 1

Zadanie 1 — przecinające się kwadraty jednostkowe (15 p.)

Zaprojektuj rekurencyjny algorytm w oparciu o strategię „dziel i zwyciężaj” rozwiązujący następujący problem. *Dane*: zbiór kwadratów na płaszczyźnie o boku jeden, których boki są równoległe do osi. *Szukane*: para przecinających się kwadratów lub odpowiedź, że takiej pary nie ma. Oblicz złożoność obliczeniową zaprojektowanego algorytmu.

Zadanie 2 — liczba etykietowań $L(2,0)$ ścieżki (15 p.)

Etykietowaniem $L(2,0)$ grafu G nazywamy funkcję przyporządkowującą wierzchołkom G liczby ze zbioru $\{1, 2, \dots, k\}$ taką, że sąsiednie wierzchołki otrzymują etykiety różniące się o co najmniej 2. Zaprojektuj wielomianowy algorytm programowania dynamicznego zliczający liczbę etykietowań $L(2,0)$ czterema etykietami ($k = 4$) dla ścieżki o n wierzchołkach. Oszacuj asymptotyczną złożoność (w terminach $\mathcal{O}(\cdot)$).

Zadanie 3 — najmniejsza transversala (15 p.)

Niech $\mathcal{S}$ będzie rodziną pewnych podzbiorów zbioru U . *Transwersalą* dla rodziny $\mathcal{S}$ nazywamy dowolny podzbiór U przecinający każdy zbiór z $\mathcal{S}$. Podaj algorytm aproksymacyjny dla problemu znajdowania najmniejszej transversali przy założeniu, że każdy element zbioru U należy do co najwyżej dwóch zbiorów z $\mathcal{S}$. *Wskazówka*: sprowadź ten problem do problemu pokrycia zbioru.

Zadanie 4 — ze schematu prawie-optimalnego do 2-aproksymacji (15 p.)

Niech P będzie problemem optymalizacyjnym, w którym poszukujemy rozwiązania dopuszczalnego maksymalizującego funkcję celu f , której zbiorem wartości jest przedział $[3, \infty)$. Wiadomo, że dla problemu P i każdego $\epsilon > 0$ istnieje algorytm wielomianowy, który dla wejścia I znajduje rozwiązanie dopuszczalne $A(I)$ takie, że $f(A(I)) \geq (1 - \epsilon)f(OPT(I)) - 1$, gdzie $OPT(I)$ to rozwiązanie optymalne. Udowodnij, że istnieje wielomianowy algorytm 2-aproksymacyjny dla problemu P .

Termin 2

Zadanie 1 — scalanie list (15 p.)

- (i) (3p.) Niech A, B będą listami długości k i l posortowanymi rosnąco. Podaj algorytm działający w czasie $\mathcal{O}(k+l)$, który scala je w jedną posortowaną rosnąco listę długości $k+l$.
- (ii) (4p.) Mamy m list długości n każda, wszystkie posortowane rosnąco; scalamy je w jedną długości mn , używając wielokrotnie algorytmu z (i): scalamy dwie pierwsze listy, wynik z trzecią, ten z czwartą, itd. Oszacuj złożoność jako funkcję m i n .
- (iii) (8p.) Podaj asymptotycznie szybszy algorytm typu „dziel i zwyciężaj” rozwiązujący problem z (ii). Oszacuj jego złożoność.

Zadanie 2 — brak słów kodowych długości 1 (15 p.)

Załóżmy, że w tekście, który chcemy zakodować za pomocą kodu Huffmana, każdy znak występuje mniej niż $\frac{24}{3}$ razy. Udowodnij, że w kodzie Huffmana dla takiego tekstu żaden znak nie będzie zakodowany za pomocą tylko jednego bitu.

Zadanie 3 — pakowanie do skrzyń (Next-Fit) jest 2-aproksymacyjne (15 p.)

Rozważmy następujący algorytm dla problemu pakowania do skrzyń. Wejściem jest nieposortowana lista rozmiarów pakowanych obiektów. Algorytm pakuje obiekty w kolejności z listy, próbując umieścić kolejny obiekt tylko w ostatnio rozpoczętej skrzyni; jeśli to niemożliwe, rozpoczyna nową skrzynię. Udowodnij, że jest to algorytm 2-aproksymacyjny dla problemu pakowania do skrzyń.

Zadanie 4 — najdłuższy wspólny podciąg kolejnych wyrazów (15 p.)

Dla zadanych ciągów $(x_1, x_2, \dots, x_n)$ i $(y_1, y_2, \dots, y_m)$ chcemy znaleźć długość ich najdłuższego wspólnego podciągu *kolejnych* wyrazów, czyli największe k , dla którego istnieją indeksy i, j takie, że $(x_i, x_{i+1}, \dots, x_{i+k-1}) = (y_j, y_{j+1}, \dots, y_{j+k-1})$. Podaj algorytm programowania dynamicznego o złożoności $\mathcal{O}(mn)$ rozwiązujący ten problem.

Termin 3**Zadanie 1 — kod Huffmana dla częstości będących potęgami dwójki (15 p.)**

Alfabet A składa się ze znaków $z_1, z_2, \dots, z_n$. W pewnym tekście T znak z_i występuje 2^{i-1} razy dla wszystkich $i \in \{1, 2, \dots, n\}$. Znajdź kod Huffmana dla tekstu T .

Zadanie 2 — rozdział czekolady (15 p.)

Kostka czekolady o wadze $2n$ dkg spadła na podłogę i połamano się na wiele kawałków, z których żaden nie waży więcej niż 1 dkg. Zaprojektuj algorytm o liniowej złożoności obliczeniowej, który rozdzieli te kawałki pomiędzy n dzieci tak, że każde dziecko dostaje kawałki o sumarycznej wadze co najmniej 1 dkg. Udowodnij, że Twój algorytm jest poprawny.

Zadanie 3 — stacje przekaźnikowe (DP) (15 p.)

Firma „Głuchy Telefon” buduje stacje przekaźnikowe wzdłuż autostrady A44 o długości M km. Jest n możliwych lokalizacji, leżących w odległościach $m_1 < m_2 < \dots < m_n$ od początku autostrady (wszystkie przy autostradzie). Koszt budowy stacji w lokalizacji i -tej to $k_i > 0$, $i \in \{1, \dots, n\}$. Stacje trzeba wybudować tak, by z każdego punktu autostrady było nie dalej niż k km do najbliższej stacji. Podaj efektywny algorytm programowania dynamicznego obliczający najmniejszy całkowity koszt inwestycji i oszacuj jego złożoność obliczeniową.

Zadanie 4 — obiekt majoryzujący (dziel i zwyciężaj) (15 p.)

Tablica A zawiera n obiektów, które nie są liczbami (np. małe pliki tekstowe). Nie jest dla nich zdefiniowana relacja $>$, ale zdefiniowana jest równość (sprawdzana w czasie stałym), oznaczająca identyczność obiektów. Mówimy, że tablica A ma *obiekt majoryzujący* M , jeśli więcej niż połowa obiektów jest równa M . Podaj algorytm typu „dziel i zwyciężaj” o złożoności $\mathcal{O}(n \log n)$, który rozstrzyga, czy A zawiera obiekt majoryzujący, i jeśli tak — zwraca go.

2018 — rozwiązania

Egzamin zerowy

Zadanie 1 — dwa najtańsze symbole pod wspólnym rodzicem (15 p.)

Udowodnij, że w kodowaniu Huffmana istnieje optymalne drzewo kodowe, w którym dwa najtańsze (najrzadziej występujące) symbole mają wspólnego rodzica.

To jest dokładnie Lemat 3.7 z wykładu; przepisujemy jego dowód poniżej

Dowód. Znajdziemy drzewo reprezentujące optymalny kod prefiksowy, w którym liście reprezentujące x i y mają wspólnego rodzica i mają największą głębokość w drzewie.

Niech T będzie dowolnym drzewem reprezentującym optymalny kod prefiksowy i niech a i b będą dwoma liśćmi o tym samym rodzicu i najmniejszej głębokości w T . Bez straty ogólności możemy przyjąć, że $f(x) \leq f(y)$ i $f(a) \leq f(b)$. $f(x)$ i $f(y)$ są najmniejszymi kluczami, więc $f(x) \leq f(a)$ i $f(y) \leq f(b)$.

Niech T' będzie drzewem powstałym z T przez zamianę liści x i a .

$$\begin{aligned} B(T) - B(T') &= \sum_{c \in C} f(c) \cdot d_T(c) - \sum_{c \in C} f(c) \cdot d_{T'}(c) = \\ &= f(a) \cdot d_T(a) + f(x) \cdot d_T(x) - f(a) \cdot d_{T'}(a) - f(x) \cdot d_{T'}(x) = \\ &= f(a) \cdot d_T(a) + f(x) \cdot d_T(x) - f(a) \cdot d_T(x) - f(x) \cdot d_T(a) = \\ &= f(a)(d_T(a) - d_T(x)) - f(x)(d_T(a) - d_T(x)) = \\ &= \underbrace{(f(a) - f(x))}_{\geq 0} \underbrace{(d_T(a) - d_T(x))}_{\geq 0} \geq 0 \end{aligned}$$

Zatem $B(T) \geq B(T')$, ale $B(T)$ najmniejsze możliwe, więc $B(T') = B(T)$ czyli T' też reprezentuje optymalny kod prefiksowy. Analogicznie zamieniamy miejscami liście y i b i otrzymujemy drzewo T'' reprezentujące optymalny kod prefiksowy, które ma żądane własności. $\square$

Zadanie 2 — skojarzenie najliczniejsze a ścieżki powiększające (15 p.)

Udowodnij, że skojarzenie w grafie jest najliczniejsze wtedy i tylko wtedy, gdy nie istnieje ścieżka powiększająca.

Jest to dokładnie twierdzenie Berge'a (twierdzenie 5.3 z notatek). Przepiszemy dowód poniżej.

Dowód. Niech M będzie skojarzeniem w G .

$\Rightarrow$ Przypuśćmy, że istnieje ścieżka powiększająca P względem M .

Wtedy $M' = M \div E(P)$ jest skojarzeniem i $|M'| > |M|$, więc M nie jest skojarzeniem najliczniejszym.

$\Leftarrow$ Przypuśćmy, że M nie jest najliczniejszym skojarzeniem w G .

Niech M' będzie skojarzeniem takim, że $|M'| > |M|$.

Niech H będzie podgrafem G indukowanym przez $M' \div M$, czyli $H = (V(G), M' \div M)$. M, M' to skojarzenia, więc stopień wierzchołka $d_H(v) \leq 2$ dla każdego $v \in V(G)$.

Każda składowa H jest ścieżką lub cyklem.

M, M' to skojarzenia, więc nie ma cykli nieparzystych w H .

Każda składowa H jest ścieżką lub cyklem parzystym.

Skoro $|M'| > |M|$, to istnieje składowa H , która jest ścieżką zawierającą więcej krawędzi z M' niż z M , więc ta ścieżka zaczyna się i kończy krawędzią z M' . Ta ścieżka jest ścieżką powiększającą względem M . $\square$

Zadanie 3 — matroid jednego wierzchołka na składową (15 p.)

Niech G będzie grafem, a $\mathcal{A}_G$ rodziną podzbiorów $V(G)$ taką, że $S \in \mathcal{A}_G$ wtedy i tylko wtedy, gdy każda składowa spójności G zawiera co najwyżej jeden wierzchołek z S . Rozstrzygnij, czy para $(V(G), \mathcal{A}_G)$ zawsze jest matroidem.

Tak — jest to *partition matroid*.

Dygresja. Partition matroid (matroid podziałowy) powstaje z podziału zbioru bazowego S na rozłączne klasy $S = C_1 \cup \dots \cup C_p$ i liczb $d_1, \dots, d_p \geq 0$ (pojemności): zbiór $A \subseteq S$ jest niezależny $\iff |A \cap C_i| \leq d_i$ dla każdej klasy i . Innymi słowy — z każdej klasy wolno wziąć co najwyżej d_i elementów, niezależnie od pozostałych klas. Że to faktycznie matroid, sprawdzamy poniżej; tutaj wszystkie $d_i = 1$, a klasami są składowe spójności G .

Suma prosta matroidów jednorodnych. Każda klasa C_i z pojemnością d_i to osobny *uniform matroid* $U_{|C_i|}^{d_i}$ (niezależne są zbiory $\leq d_i$ elementów), a cały partition matroid jest ich sumą prostą. W szczególności *uniform matroid* to przypadek jednej klasy $C_1 = S$ z pojemnością r , oznaczany U_n^r ($n = |S|$).

Baza, cykl, rząd. Baza bierze z każdego bloku dokładnie d_i elementów; cykl (minimalny zbiór zależny) to $d_i + 1$ elementów z jednego bloku; rząd matroidu to $\sum_i d_i$. (Dla naszego G : baza = jeden wierzchołek na składową, rząd = liczba składowych.)

Powiązania. Klasa matroidów podziałowych jest zamknięta na dualność, minory i sumy prostych. Gdy wszystkie $d_i = 1$, partition matroid jest zarazem *transversal matroid* rodziny rozłącznych bloków; na tym opiera się ujęcie skojarzeń w grafie dwudzielnym (U, V) jako przecięcia dwóch matroidów (po jednym „brak wspólnego końca” dla strony U i dla V).

Niech $C_1, \dots, C_p$ będą składowymi spójności G ; tworzą one podział $V(G)$. Z definicji $S \in \mathcal{A}_G \iff |S \cap C_i| \leq 1$ dla każdego i . Sprawdzamy aksjomaty:

(wł. 1) $V(G)$ jest skończony i niepusty. ✓

(wł. 2) $\emptyset \in \mathcal{A}_G$, więc $\mathcal{A}_G \neq \emptyset$. ✓

(wł. 3) Jeśli $S \in \mathcal{A}_G$ i $S' \subseteq S$, to $|S' \cap C_i| \leq |S \cap C_i| \leq 1$, więc $S' \in \mathcal{A}_G$.

(wł. 4) Niech $S_1, S_2 \in \mathcal{A}_G$, $|S_1| < |S_2|$. Każda składowa zawiera ≤ 1 element każdego ze zbiorów, więc $|S_j|$ równa się liczbie składowych „trafionych” przez S_j . Z $|S_2| > |S_1|$ wynika, że istnieje składowa C_i trafiona przez S_2 , ale nie przez S_1 ; biorąc $v \in S_2 \cap C_i$ mamy $S_1 \cup \{v\} \in \mathcal{A}_G$.

Zatem $(V(G), \mathcal{A}_G)$ jest matroidem. □

Zadanie 4 — dokładne pokrycie wierzchołkowe w czasie podwykładniczym (15 p.)

Zaprojektuj algorytm dokładny, który znajdzie licznosc najmniejszego pokrycia wierzchołkowego grafu w czasie $o(2^n)$, gdzie n jest liczba wierzchołków.

Rozgałęzianie po wierzchołku dużego stopnia. Korzystamy z obserwacji: dla dowolnej krawędzi pokrycie C musi zawierać jeden z jej końców, a równoważnie — dla dowolnego wierzchołka v albo $v \in C$, albo (jeśli $v \notin C$) wszyscy jego sąsiedzi $N(v) \subseteq C$.

Algorytm 1: Minimalne pokrycie wierzchołkowe przez rozgałęzianie

1: function MINVC(G)	▷ zwraca licznosc najmniejszego pokrycia
2: if G nie ma krawędzi then	
3: return 0	
4: if maksymalny stopień w G wynosi ≤ 2 then	
5: return minimalne pokrycie sumy ścieżek i cykli	▷ DP po każdej ścieżce/cyklu czas wielomianowy
6: wybierz wierzchołek v o stopniu ≥ 3	
7: $a \leftarrow 1 + \text{MINVC}(G - v)$	▷ gałąź $v \in C$
8: $b \leftarrow N(v) + \text{MINVC}(G - v - N(v))$	▷ gałąź $v \notin C$, więc $N(v) \subseteq C$
9: return $\min(a, b)$	

Dygresja. DP (*Dynamic Programming*) po ścieżce/cyklu. Przy maksymalnym stopniu ≤ 2 graf rozpada się na rozłączne ścieżki i cykle; każdą składową liczymy osobno i sumujemy wyniki. Dla ścieżki $v_1, \dots, v_k$ trzymamy dwa stany:

- $f[i][0]$ — minimalne pokrycie wierzchołków $v_1, \dots, v_i$ przy $v_i \notin C$;
- $f[i][1]$ — to samo przy $v_i \in C$.

Każda krawędź $v_{i-1}v_i$ musi być pokryta, stąd przejścia

$$f[i][0] = f[i-1][1], \quad f[i][1] = 1 + \min(f[i-1][0], f[i-1][1]).$$

Jeśli $v_i \notin C$, to poprzednik musi być w C (inaczej krawędź $v_{i-1}v_i$ niepokryta); jeśli $v_i \in C$, poprzednik dowolny. Baza to $f[1][0] = 0$, $f[1][1] = 1$, a wynik ścieżki to $\min(f[k][0], f[k][1])$.

Dla cyklu dochodzi jedna domykająca krawędź v_kv_1 . Robimy ten sam rachunek co dla ścieżki $v_1, \dots, v_k$, ale uruchamiamy go dwukrotnie, ustalając z góry stan v_1 — różnica jest tylko w *bazie* (w v_1) i w *odczycie* (w v_k):

- *wymuszamy* $v_1 \in C$: baza $f[1][1] = 1$, $f[1][0] = +\infty$; krawędź v_kv_1 jest pokryta przez v_1 , więc v_k dowolny — odczyt $\min(f[k][0], f[k][1])$;
- *wymuszamy* $v_1 \notin C$: baza $f[1][0] = 0$, $f[1][1] = +\infty$; teraz v_kv_1 wymusza $v_k \in C$ — odczyt $f[k][1]$.

(Wartość $+\infty$ zakazuje stanu sprzecznego z wymuszeniem.) Każda składowa w czasie liniowym, więc cały ten przypadek jest wielomianowy.

Poprawność. Przy stopniu ≥ 3 każde pokrycie wpada do jednej z dwóch gałęzi (zawiera v lub całe $N(v)$), a obie gałęzie zwracają poprawne minimum dla swojej podinstancji; bierzemy mniejsze. Przypadek maksymalnego stopnia ≤ 2 rozwiązujemy wprost (graf jest sumą ścieżek i cykli).

Złożoność. Gałąź pierwsza usuwa 1 wierzchołek, druga co najmniej $1 + 3 = 4$ (sam v oraz ≥ 3 jego sąsiadów). Składnik $\text{poly}(n)$ to praca poza wywołaniami rekurencyjnymi: znalezienie wierzchołka o stopniu ≥ 3 (lub stwierdzenie, że maksymalny stopień ≤ 2), zbudowanie podgrafów $G-v$ oraz $G-v-N(v)$ i DP po ścieżkach/cyklach w przypadku bazowym — każda z tych operacji jest wielomianowa względem liczby wierzchołków i krawędzi. Stąd

$$T(n) \leq T(n-1) + T(n-4) + \text{poly}(n).$$

Równanie charakterystyczne $x^4 = x^3 + 1$ ma pierwiastek dodatni $\alpha \approx 1,3803$, więc $T(n) = \mathcal{O}(\alpha^n) = \mathcal{O}(1,39^n) = o(2^n)$. $\square$

Zadanie 5 — 3-aproksymacja problemu 3DC (15 p.)

Rozważmy problem — nazwijmy go roboczo $3DC$ — w którym mamy daną rodzinę $\mathcal{R}$ 3-elementowych podzbiorów pewnego zbioru X i poszukujemy jak najmniej licznego zbioru $S \subseteq X$ takiego, że każdy zbiór z $\mathcal{R}$ zawiera przynajmniej jeden element z S . Zaprojektuj wielomianowy algorytm 3-aproksymacyjny dla problemu 3DC (i udowodnij, że stała aproksymacji istotnie jest nie większa niż 3). Możesz zainspirować się algorytmem 2-aproksymacyjnym dla problemu najmniejszego pokrycia wierzchołkowego.

Algorytm (jak 2-aproksymacja dla pokrycia wierzchołkowego).

Algorytm 2: 3-aproksymacja problemu 3DC przez maksymalne dopasowanie

```

1: function APPROX3DC( $X, \mathcal{R}$ )
2:    $S \leftarrow \emptyset$ 
3:   while istnieje  $R \in \mathcal{R}$  rozłączny z  $S$  do
4:     wybierz taki  $R$ 
5:      $S \leftarrow S \cup R$  ▷ dodaj wszystkie trzy elementy
6:   return  $S$ 
    
```

Po zakończeniu każdy zbiór z $\mathcal{R}$ jest przecięty przez S , więc S jest poprawnym rozwiązaniem.

Współczynnik aproksymacji. Niech $R_1, \dots, R_t$ będą zbiorami wybranymi w kroku 2. Są one parami rozłączne: gdy wybieramy R_i , jest on rozłączny z bieżącym S , a ten zawiera wszystkie elementy wcześniej wybranych $R_1, \dots, R_{i-1}$; zatem R_i jest rozłączny z każdym R_j , $j < i$. Każde rozwiązanie optymalne S^* musi przecinać każdy R_i , a skoro zbiory R_i są rozłączne, potrzebuje na to co najmniej t różnych elementów: $|S^*| \geq t$. Nasze rozwiązanie ma $|S| = 3t \leq 3|S^*|$. Algorytm jest więc 3-aproksymacyjny.

Złożoność. Każdy obrót pętli dokłada do S trzy nowe elementy (wybrany R jest rozłączny z S), więc pętla wykonuje się co najwyżej $\lfloor |X|/3 \rfloor$ razy. W każdym obrocie przeglądamy rodzinę $\mathcal{R}$ w poszukiwaniu zbioru rozłącznego z S , sprawdzając dla każdego $R \in \mathcal{R}$ jego trzy elementy — to $O(|\mathcal{R}|)$ pracy. Łącznie $O(|X| \cdot |\mathcal{R}|)$, czyli czas wielomianowy. $\square$

Termin 1

Zadanie 1 — wszystkie liście Huffmana na tej samej głębokości (15 p.)

Niech T będzie optymalnym drzewem Huffmana dla $|A| = 2^k$ symboli, przy czym dla każdych trzech symboli $f(x) + f(y) > f(z)$. Udowodnij, że wszystkie liście T mają głębokość k .

Warunek „ $f(x) + f(y) > f(z)$ dla wszystkich trójek” oznacza, że suma dwóch najmniejszych częstości przewyższa największą. Pokażemy indukcyjnie po k , że algorytm Huffmana buduje pełne drzewo binarne głębokości k .

Krok bazowy. Dla $k = 0$ mamy jeden symbol, drzewo to pojedynczy liść na głębokości 0.

Krok indukcyjny. Niech $n = 2^k$ i $f_1 \leq f_2 \leq \dots \leq f_n$. Załóżmy $f_1 + f_2 > f_n$.

Najpierw kojarzone są same liście. Algorytm łączy dwie najmniejsze częstości w węzeł $w = f_1 + f_2$. Z założenia $w = f_1 + f_2 > f_n \geq f_i$ dla każdego oryginalnego liścia, więc każdy nowo utworzony węzeł wewnętrzny ma wagę większą niż dowolny pozostały liść. W konsekwencji, dopóki istnieją niepołączone oryginalne liście, dwie najmniejsze wagi w kolejce są zawsze liśćmi — algorytm najpierw paruje wszystkie 2^k liści w 2^{k-1} węzłów, tworząc kompletny najniższy poziom.

Nowe wagi spełniają ten sam warunek. Po sparowaniu mamy 2^{k-1} wag $w_1, \dots, w_{2^{k-1}}$, każda będąca sumą pewnej pary liści, więc $w_i \geq f_1 + f_2 > f_n$. Każda waga to suma dwóch liści, więc największa $\leq f_{n-1} + f_n$. Stąd suma dwóch najmniejszych nowych wag spełnia

$$w_i + w_j > 2f_n \geq f_{n-1} + f_n \geq \max_{\ell} w_{\ell},$$

czyli warunek „suma dwóch najmniejszych $>$ największa” zachodzi dla nowej rodziny 2^{k-1} wag.

Z założenia indukcyjnego algorytm zbuduje na tych wagach pełne drzewo głębokości $k - 1$; doczepienie pod każdym jego liściem pary liści oryginalnych daje pełne drzewo głębokości k . Wszystkie liście T leżą więc na głębokości k . $\square$

Zadanie 2 — najtańszy element w bazie minimalnej (15 p.)

Matroid $M = (S, \mathcal{I})$, wagi $w : S \rightarrow \mathbb{R}_+$, $S \ni x$ minimalizuje $w(x)$. Udowodnij, że x należy do pewnej bazy o minimalnej wadze (innymi słowy: istnieje zbiór $I \in \mathcal{I}$ taki, że $x \in I$ oraz $w(i) = \min_{J \in \mathcal{I}} w(J)$).

Zauważmy, że x nie jest pętlą, tzn. $\{x\} \in \mathcal{I}$. Gdyby bowiem $\{x\}$ było zależne, to $\{x\}$ nie zawierałoby się w żadnym zbiorze niezależnym, a więc x nie należałoby do żadnej bazy i teza byłaby fałszywa. Tezę rozumiemy zatem przy założeniu, że minimalizujący wagę element nie jest pętlą (pętle, jako nieobecne w żadnej bazie, można usunąć z S).

Dygresja. Pętlą w matroidzie $M = (S, \mathcal{I})$ nazywamy element x , dla którego już sam jednoelementowy zbiór jest zależny: $\{x\} \notin \mathcal{I}$ (równoważnie $\text{rank}(\{x\}) = 0$). Skoro żaden zbiór niezależny nie może zawierać pętli, nie zawiera jej też żadna baza — pętla nigdy nie wejdzie do rozwiązania, więc spokojnie pomijamy ją w rozważaniach. Nazwa pochodzi od matroidu grafowego: krawędź będąca pętlą (łączyącą wierzchołek z samym sobą) sama tworzy cykl, więc nie należy do żadnego lasu.

W matroidzie liniowym (gdzie S to zbiór wektorów, a niezależność to liniowa niezależność) pętlą jest wektor zerowy: $\{0\}$ jest zależny, bo $1 \cdot 0 = 0$, więc 0 nie wchodzi do żadnej bazy.

Rozwiązanie (algorytm zachłanny). Algorytm zachłanny dla matroidów (Rado–Edmonds, Twierdzenie 3.20) sortuje elementy rosnąco po wadze i kolejno dokłada do budowanego zbioru każdy element, który zachowuje niezależność; zwracany zbiór jest bazą o minimalnej wadze. Element x ma najmniejszą wagę, więc przy odpowiednim porządku jest rozważany jako pierwszy, a ponieważ $\{x\} \in \mathcal{I}$, zostaje dodany do budowanej bazy. Zatem zwrócona baza ma minimalną wagę i zawiera x , co dowodzi tezy. $\square$

Alternatywnie (dla ciekawych) — argument wymiany przez obwody. Poniższy dowód nie odwołuje się do poprawności algorytmu zachłannego, lecz wprost przekształca dowolną bazę minimalną tak, by zawierała x . Wymaga jednak pojęcia obwodu, którego nie wprowadzaliśmy na wykładzie.

Obwody. Obwodem (cyklem) matroidu nazywamy minimalny zbiór zależny: $\mathcal{C}$ jest obwodem, gdy $\mathcal{C} \notin \mathcal{I}$, ale każdy jego właściwy podzbiór jest niezależny. W matroidzie grafowym są to dokładnie zwykłe cykle grafu, stąd nazwa.

Dygresja. Skorzystamy z aksjomatu eliminacji obwodów: jeśli $\mathcal{C}_1 \neq \mathcal{C}_2$ są obwodami i $e \in \mathcal{C}_1 \cap \mathcal{C}_2$, to istnieje obwód $\mathcal{C}_3 \subseteq (\mathcal{C}_1 \cup \mathcal{C}_2) \setminus \{e\}$. W matroidzie grafowym: dwa cykle dzielące krawędź e sklejone wzdłuż e i pozbawione e wciąż zawierają jakiś cykl. Aksjomat ten (wraz z tym, że $\emptyset$ nie jest obwodem i żaden obwód nie zawiera się w innym) jest równoważną charakterystyką matroidu przez rodzinę obwodów.

Wykorzystamy dwie własności:

- jeśli $I \in \mathcal{I}$, a $I \cup \{x\} \notin \mathcal{I}$, to $I \cup \{x\}$ zawiera *dokładnie jeden* obwód, i przechodzi on przez x . *Istnienie:* każdy zbiór zależny zawiera obwód (jego minimalny zależny podzbiór), a ten musi zawierać x — inaczej leżałby w niezależnym I . *Jedyność:* gdyby były dwa różne obwody $\mathcal{C}_1, \mathcal{C}_2 \subseteq I \cup \{x\}$, oba zawierałyby x , więc z aksjomatu eliminacji obwodów (dla $x \in \mathcal{C}_1 \cap \mathcal{C}_2$ istnieje obwód $\mathcal{C}_3 \subseteq (\mathcal{C}_1 \cup \mathcal{C}_2) \setminus \{x\}$) dostalibyśmy obwód zawarty w I — sprzeczność z $I \in \mathcal{I}$;
- (wymiana) jeśli $\mathcal{C}$ jest obwodem i $y \in \mathcal{C}$, to usunięcie y z tego zbioru zależnego przywraca niezależność — to wprost definicja: $\mathcal{C} \setminus \{y\}$ jest właściwym podzbiorem obwodu, więc jest niezależny.

Argument wymiany. Niech B będzie dowolną bazą o minimalnej wadze. Jeśli $x \in B$, gotowe. W przeciwnym razie $B \cup \{x\}$ zawiera dokładnie jeden obwód (cykl) $\mathcal{C}$ przechodzący przez x . Obwód ma co najmniej dwa elementy, więc istnieje $y \in \mathcal{C} \setminus \{x\} \subseteq B$. Z własności wymiany dla obwodów zbiór

$$B' = (B \setminus \{y\}) \cup \{x\}$$

jest bazą (usunięcie dowolnego elementu obwodu po dodaniu x przywraca niezależność i zachowuje licznosc). Ponieważ x ma najmniejszą wagę, $w(x) \leq w(y)$, więc

$$w(B') = w(B) + w(x) - w(y) \leq w(B).$$

Skoro B jest minimalna, $w(B') = w(B)$, czyli B' jest bazą minimalnej wagi zawierającą x . $\square$

Zadanie 3 — mnożenie macierzy z 32 mnożeniami bloków (15 p.)

Dane są wzory mnożące dwie macierze 4×4 za pomocą 32 mnożeń i 256 dodawań liczb.^a Skonstruuj rekurencyjny algorytm mnożenia macierzy $n \times n$ i wyznacz jego złożoność.

^aZakładamy również, że te wzory nie korzystają z przemienności mnożenia.

Algorytm. Dla $n = 4^m$ dzielimy każdą macierz $n \times n$ na siatkę 4×4 bloków rozmiaru $\frac{n}{4} \times \frac{n}{4}$. Stosujemy dane wzory traktując bloki jak „skalary”: ponieważ wzory nie korzystają z przemienności mnożenia, są poprawne także dla nieprzemiennego mnożenia bloków (macierzy). Wymaga to:

- 32 mnożeń bloków $\frac{n}{4} \times \frac{n}{4}$ (rekurencyjnie),
- 256 dodawaniań/odejmowań bloków, każde w czasie $\mathcal{O}((n/4)^2) = \mathcal{O}(n^2)$.

Złożoność.

$$T(n) = 32 \cdot T\left(\frac{n}{4}\right) + \mathcal{O}(n^2)$$

$$a = 32, \quad b = 4, \quad k = 2$$

$$32 > 4^2 \Rightarrow T(n) = \Theta(n^{\log_4 32}) = \Theta(n^{2,5})$$

$$\text{bo } \log_4 32 = \frac{\log_2 32}{\log_2 4} = \frac{5}{2} = 2,5. \quad \square$$

Zadanie 4 — minimalny podział grafu na trójkąty (15 p.)

Przez *podział grafu G na trójkąty* rozumiemy podział zbioru $V(G)$ na trzyelementowe podzbiory $\{u_1, v_1, w_1\}, \{u_2, v_2, w_2\}, \dots, \{u_k, v_k, w_k\}$, gdzie dla każdego i zachodzi $u_i v_i, v_i w_i, u_i w_i \in E(G)$. Jeżeli $w: E(G) \rightarrow \mathbb{R}$, to przez *wagę* takiego podziału rozumiemy sumę wag krawędzi wewnątrz klas podziału, tzn. wartość wyrażenia $\sum_{i=1}^k (w(u_i v_i) + w(v_i w_i) + w(u_i w_i))$.

W problemie MINIMUM TRIANGLE PARTITION mamy dany graf ważony krawędziowo i poszukujemy minimalnej wagi podziału tego grafu na trójkąty. Zaprojektuj algorytm, który rozwiąże ten problem w czasie $\mathcal{O}^*(2^n)$, gdzie n jest liczbą wierzchołków grafu; uzasadnij poprawność i oszacuj czas działania swojego algorytmu.

Programowanie dynamiczne po podzbiorach. Dla $U \subseteq V(G)$ niech $D[U]$ będzie minimalną wagą podziału U na trójkąty ($D[U] = \infty$, gdy $|U|$ nie dzieli się przez 3 lub podział nie istnieje). Ustalmy w U wierzchołek o najmniejszym indeksie $v = \min U$. W każdym podziale v należy do dokładnie jednego trójkąta $\{v, a, b\}$, gdzie $a, b \in U$ oraz $va, vb, ab \in E(G)$. Stąd

$$D[\emptyset] = 0, \quad D[U] = \min_{\substack{a, b \in U \setminus \{v\} \\ \{v, a, b\} \text{ — trójkąt}}} \left(w(va) + w(vb) + w(ab) + D[U \setminus \{v, a, b\}] \right).$$

Odpowiedzią jest $D[V(G)]$.

Algorytm 3: Minimalny podział grafu na trójkąty

```

1: function MINTRIANGLEPARTITION( $G, w$ ) ▷ zwraca  $D[V(G)]$ 
2:    $D[\emptyset] \leftarrow 0$ 
3:   for  $U \subseteq V(G)$  rosnąco po  $|U|$ ,  $U \neq \emptyset$  do
4:      $D[U] \leftarrow \infty$ 
5:     if  $|U| \bmod 3 \neq 0$  then
6:       continue
7:      $v \leftarrow \min U$ 
8:     for  $a, b \in U \setminus \{v\}$ ,  $a < b$  do
9:       if  $va, vb, ab \in E(G)$  then ▷  $\{v, a, b\}$  jest trójkątem
10:         $D[U] \leftarrow \min(D[U], w(va) + w(vb) + w(ab) + D[U \setminus \{v, a, b\}])$ 
11:   return  $D[V(G)]$ 

```

Poprawność. Ustalenie najmniejszego $v \in U$ usuwa nadmiarowość: każdy podział U ma dokładnie jeden trójkąt zawierający v (ten, do którego v należy), więc rekurencja generuje każdy podział dokładnie raz — nie rozpatrujemy tego samego podziału wielokrotnie, raz „od trójkąta $\{v, a, b\}$ ”, innym razem od innej klasy. Reszta $U \setminus \{v, a, b\}$ jest dzielona niezależnie i optymalnie (optymalna podstruktura). Iterujemy po wszystkich możliwych trójkątach zawierających v , więc znajdujemy minimum.

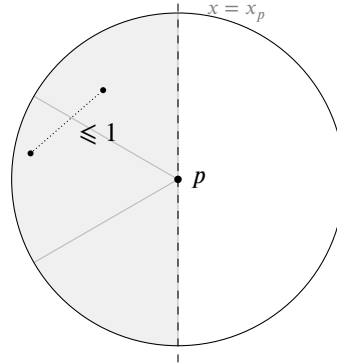
Złożoność. Podzbiorów jest 2^n ; dla każdego, przy ustalonym v , przeglądamy pary (a, b) w czasie $\mathcal{O}(n^2)$, a sprawdzenie, czy $\{v, a, b\}$ jest trójkątem (tzn. $va, vb, ab \in E(G)$), kosztuje $\mathcal{O}(1)$ przy macierzy sąsiedztwa. Łącznie $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$ czasu i $\mathcal{O}(2^n)$ pamięci. $\square$

Zadanie 5 — 3-aproksymacyjne kolorowanie grafów dysków jednostkowych (15 p.)

Podaj wielomianowy algorytm 3-aproksymacyjny dla problemu kolorowania grafów przecięć dysków jednostkowych^a i udowodnij, że współczynnik aproksymacji istotnie wynosi 3. Możesz pominąć dowód potrzebnego geometrycznego faktu (ale musisz go sformułować).

^aPrzez *graf przecięć dysków jednostkowych* rozumiemy tu graf G taki, że $V(G)$ jest skończonym podzbiorem $\mathbb{R}^2$ oraz $(x_1, y_1)(x_2, y_2) \in E(G)$ wtedy i tylko wtedy, gdy euklidesowa odległość między punktami (x_1, y_1) i (x_2, y_2) wynosi nie więcej niż 1.

Fakt geometryczny. Rozważmy koło o środku w p i promieniu 1 (zawiera ono wszystkich sąsiadów p). Jego lewą połowę — czyli punkty o współrzędnej x nie większej niż x_p — można pokryć trzema wycinkami o kącie 60° każdy. Jeżeli dwa punkty leżą w tym samym wycinku i oba są w odległości ≤ 1 od p , to są też w odległości ≤ 1 od siebie: cięciwa łuku o promieniu 1 i kącie 60° ma długość $2 \sin 30^\circ = 1$, a zatem żadne dwa takie punkty nie są dalej niż o tę cięciwę. Wobec tego sąsiedzi p leżący w jednym wycinku tworzą wraz z p klikę.



Rysunek 1: Lewa połowa koła o promieniu 1 pokryta trzema wycinkami po 60° ; punkty w jednym wycinku są wzajemnie w odległości ≤ 1 .

Algorytm. Sortujemy punkty rosnąco po współrzędnej x i kolorujemy zachłannie: każdemu p nadajemy najmniejszy kolor nieużyty przez jego wcześniej pokolorowanych sąsiadów (czyli tych o $x \leq x_p$).

Współczynnik aproksymacji. Wcześniejsi sąsiedzi p leżą w lewej półpłaszczyźnie koła, więc — na mocy faktu — rozkładają się na 3 kliki. Każda z tych klik wraz z p tworzy klikę, więc zawiera co najwyżej $\omega - 1$ wcześniejszych sąsiadów (gdzie ω to liczność największej kliki). Stąd wcześniejszych sąsiadów jest co najwyżej $3(\omega - 1)$, a algorytm zachłanny nadaje p kolor o numerze nie większym niż $3(\omega - 1) + 1 = 3\omega - 2$. Zatem liczba użytych kolorów spełnia

$$\chi_{\text{alg}} \leq 3\omega - 2.$$

Ponieważ każda klika wymaga osobnych kolorów, $\chi \geq \omega$, skąd

$$\frac{\chi_{\text{alg}}}{\chi} \leq \frac{3\omega - 2}{\omega} = 3 - \frac{2}{\omega} \leq 3.$$

Algorytm jest więc 3-aproksymacyjny (i działa w czasie $\mathcal{O}(n \log n + m)$). $\square$

Termin 2

Zadanie 1 — ścieżki powiększające zbioru niezależnego w grafie bezpazurowym (15 p.)

Mówimy, że graf G jest *bezpazurowy*, jeżeli żaden z wierzchołków G nie ma trzech niezależnych^a sąsiadów. Niech I będzie zbiorem niezależnym w grafie. Mówimy, że ścieżka $P = v_1 v_2 v_3 \dots v_k$ jest *ścieżką powiększającą* dla I , jeżeli P ma nieparzystą liczbę wierzchołków, wierzchołki $v_2, v_4, v_6, \dots$ (o parzystych indeksach) należą do I , wierzchołki $v_1, v_3, v_5, \dots$ nie należą do I oraz graf G nie zawiera żadnej krawędzi między wierzchołkami niesąsiadującymi na ścieżce. Niech G będzie grafem bezpazurowym, a I zbiorem niezależnym w G . Udowodnij, że jeżeli I nie jest najliczniejszym zbiorem niezależnym w G , to istnieje ścieżka powiększająca dla I . *Wskazówka:* zainspiruj się dowodem analogicznego stwierdzenia dla skojarzeń.

^aZbiór wierzchołków S w grafie jest *niezależny*, jeżeli żadne dwa wierzchołki z S nie są połączone krawędzią.

Analog twierdzenia Berge’a. Niech J będzie zbiorem niezależnym o $|J| > |I|$. Rozważmy $D = I \div J = (I \setminus J) \cup (J \setminus I)$ oraz podgraf indukowany $G[D]$. Krawędzie w $G[D]$ łączą tylko $I \setminus J$ z $J \setminus I$ (każdy z tych zbiorów jest niezależny).

Stopnie w $G[D]$ są ≤ 2 . Weźmy $v \in I \setminus J$. Jego sąsiady w D leżą w $J \setminus I$, a J jest niezależny, więc są one parami niesąsiednie. Gdyby v miał trzech takich sąsiadów, byłyby one trzema niezależnymi sąsiadami v — pazur, sprzeczność z bezpazurowością. Zatem $\deg_{G[D]}(v) \leq 2$; tak samo dla $v \in J \setminus I$.

Skoro $G[D]$ ma maksymalny stopień ≤ 2 , jego składowe to ścieżki i cykle, naprzemienne między $I \setminus J$ a $J \setminus I$ (bo krawędzie idą tylko między tymi stronami); cykle są parzyste. Każdy cykl i każda ścieżka parzysta ma tyle samo wierzchołków z I co z J . Ponieważ $|J \setminus I| > |I \setminus J|$, istnieje składowa z przewagą wierzchołków $J \setminus I$ — jest to ścieżka P o obu końcach w $J \setminus I$, naprzemienna, o nieparzystej liczbie wierzchołków.

Ścieżka P jest pełną składową $G[D]$, więc jest indukowana (brak krawędzi między niesąsiadującymi na niej wierzchołkami). Jej końce należą do $J \setminus I$, więc nie należą do I ; wierzchołki na pozycjach parzystych to $I \setminus J \subseteq I$, a na nieparzystych $J \setminus I \not\subseteq I$. Jest to więc ścieżka powiększająca dla I . $\square$

Zadanie 2 — matroid z ograniczeniami licznosciowymi na sumie zbiorów (15 p.)

A, B rozłączne, $S \in \mathcal{F} \iff |S \cap B| \leq k$ oraz $|S \cap A| \leq |S \cap B|$. Rozstrzygnij, czy $(A \cup B, \mathcal{F})$ jest matroidem.

Tak — jest to matroid. Dla zbioru S piszemy $a(S) = |S \cap A|$, $b(S) = |S \cap B|$; warunek niezależności to $a(S) \leq b(S) \leq k$.

- **Niepustość:** $\emptyset \in \mathcal{F}$ ($0 \leq 0 \leq k$).
- **Dziedziczność:** usunięcie elementu nie zwiększa a ani b , a zmniejszenie obu wielkości zachowuje $a \leq b \leq k$.
- **Wymiana:** niech $I_1, I_2 \in \mathcal{F}$, $|I_1| < |I_2|$. Oznaczmy $a_i = a(I_i)$, $b_i = b(I_i)$; mamy $a_1 + b_1 < a_2 + b_2$.
 - Jeśli $b_1 < k$ i $I_2 \setminus I_1$ zawiera element B , dodajemy go: nowe $b_1+1 \leq k$ oraz $a_1 \leq b_1 < b_1+1$ — OK.
 - W przeciwnym razie albo $b_1 = k$, albo $I_2 \setminus I_1$ nie ma elementów B . Pokażemy, że da się dodać element A , czyli że $a_1 < b_1$ oraz $I_2 \setminus I_1$ zawiera element A .

Rozważmy ostatni przypadek dokładniej. Jeśli $b_1 = k$, to $b_2 \leq k = b_1$, więc $a_2 + b_2 > a_1 + b_1$ daje $a_2 > a_1$; gdyby $a_1 = k = b_1$, to $|I_1| = 2k \geq a_2 + b_2 = |I_2|$ — sprzeczność, zatem $a_1 < k = b_1$. Jeśli zaś $I_2 \setminus I_1$ nie ma elementów B (a $b_1 < k$ nie zadziałało, bo wtedy poprzedni punkt by zakończył), to $b_2 \leq b_1$ i znów $a_2 > a_1$, a $a_1 \leq b_1$; gdyby $a_1 = b_1$, to z $b_2 \leq b_1$ mielibyśmy $a_2 \leq b_2 \leq b_1 = a_1$, sprzeczne z $a_2 > a_1$, więc $a_1 < b_1$. W obu sytuacjach $a_1 < b_1$ oraz $a_2 > a_1$, więc $I_2 \setminus I_1$ zawiera element A ; dodanie go daje $a_1+1 \leq b_1$ i zachowuje niezależność.

We wszystkich przypadkach istnieje element $e \in I_2 \setminus I_1$ z $I_1 \cup \{e\} \in \mathcal{F}$, więc spełniona jest własność wymiany. Para $(A \cup B, \mathcal{F})$ jest matroidem. $\square$

Zadanie 3 — para najbliższych punktów w czasie $n \log n$ (15 p.)

Podaj algorytm $\mathcal{O}(n \log n)$ znajdujący parę najbliższych punktów na płaszczyźnie; sformułuj potrzebny fakt geometryczny.

Standardowy algorytm „dziel i zwyciężaj” z wykładu o parze najbliższych punktów (rys. 1, rys. 2); tu w metryce euklidesowej.

Algorytm.

- **Dziel.** Wstępnie sortujemy punkty po x i dzielimy pionową prostą l (mediana współrzędnej x) na dwie połowy po $\lfloor n/2 \rfloor$ punktów.
- **Zwyciężaj.** Rekurencyjnie wyznaczamy najmniejsze odległości δ_L, δ_R ; $\delta = \min(\delta_L, \delta_R)$.
- **Połącz.** W pasie $|x - l| \leq \delta$, przeglądając punkty posortowane po y , porównujemy każdy punkt tylko z kolejnymi, których y różni się o $\leq \delta$.

Dla $n \leq 3$ liczymy wszystkie pary wprost.

Fakt geometryczny. W prostokącie $2\delta \times \delta$ (pas o szerokości 2δ , wysokości δ) leży co najwyżej stała liczba punktów (np. ≤ 8): gdyby było ich więcej, dwa leżałyby po tej samej stronie prostej l w odległości $< \delta$, sprzecznie z definicją δ . Dzięki temu faza łączenia kosztuje $\mathcal{O}(n)$ (każdy punkt porównuje się z $\mathcal{O}(1)$ następnikami).

Złożoność. Sortowanie $\mathcal{O}(n \log n)$; rekurencja $T(n) = 2T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n)$. Łącznie $\mathcal{O}(n \log n)$. $\square$

Zadanie 4 — minimalny podział grafu na trójkąty (15 p.)MINIMUM TRIANGLE PARTITION w czasie $\mathcal{O}^*(2^n)$.

Identyczne z rozwiązaniem zadania 4 z Terminu 1 (podrozdz. A.2): programowanie dynamiczne $D[U] = \min \text{ po trójkątach zawierających min } U \text{ z } w(\text{trójkąt}) + D[U \setminus \text{trójkąt}]$, w czasie $\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n)$. $\square$

Zadanie 5 — 3-aproksymacyjne kolorowanie grafów dysków jednostkowych (15 p.)

Wielomianowy algorytm 3-aproksymacyjny dla kolorowania grafów dysków jednostkowych.

Identyczne z rozwiązaniem zadania 5 z Terminu 1 (podrozdz. A.2): zachłanne kolorowanie w kolejności rosnącej po x ; lewa półpłaszczyzna koła rozkłada się na 3 wycinki-kliki, więc liczba użytych kolorów to $\leq 3\omega - 2 \leq 3\chi$. $\square$

Termin poprawkowy

Zadanie 1 — niemalejące długości najkrótszych ścieżek powiększających (15 p.)

P — najkrótsza ścieżka powiększająca dla M , $M' = M \div P$, P' — najkrótsza ścieżka powiększająca dla M' . Udowodnij $|P| \leq |P'|$ oraz że wspólny wierzchołek wymusza $|P| < |P'|$.

(Lemat z algorytmu Hopcrofta–Karpa.) Mierzmy $|P|$ liczbą krawędzi.

Nierówność $|P| \leq |P'|$. Niech $M'' = M' \div P'$; wtedy $|M''| = |M| + 2$. Rozważmy $D = M \div M''$. Jako symetryczna różnica dwóch skojarzeń, D ma stopnie ≤ 2 , więc rozpada się na ścieżki i cykle naprzemienne względem M . Skoro $|M''| = |M| + 2$, w D jest o dwie krawędzie M'' więcej niż M , zatem D zawiera co najmniej dwie wierzchołkowo-rozłączne ścieżki powiększające Q_1, Q_2 względem M . Każda z nich ma długość $\geq |P|$ (bo P jest najkrótsza), więc

$$|D| \geq |Q_1| + |Q_2| \geq 2|P|.$$

Z drugiej strony

$$D = M \div M'' = (M \div M') \div (M' \div M'') = P \div P',$$

a dla symetrycznej różnicy zbiorów krawędzi $|P \div P'| = |P| + |P'| - 2c$, gdzie $c = |E(P) \cap E(P')|$ to liczba wspólnych krawędzi. Łącząc:

$$|P| + |P'| - 2c \geq 2|P| \implies |P'| \geq |P| + 2c \geq |P|.$$

Wspólny wierzchołek $\Rightarrow$ ostra nierówność. Pokażemy kontrapozycję: jeśli $|P'| = |P|$, to P i P' są wierzchołkowo rozłączne. Z powyższego $|P'| = |P|$ wymusza $c = 0$ (brak wspólnych krawędzi), więc $D = P \div P'$ jest rozłączną krawędziowo sumą P i P' . Ale D ma stopnie ≤ 2 :

- gdyby P i P' miały wspólny wierzchołek wewnętrzny v , jego stopień w D wynosiłby $2 + 2 = 4 > 2$ — sprzeczność;
- wspólnym wierzchołkiem nie może też być koniec: końce P są wolne względem M , lecz skojarzone względem $M' = M \div P$, więc nie są wolnymi końcami P' (a końce P' muszą być wolne względem M'); końce P' z kolei mają w D stopień 1, a wierzchołek wewnętrzny P ma stopień 2, co znów dałoby stopień 3.

Zatem P, P' są wierzchołkowo rozłączne. Równoważnie: wspólny wierzchołek $\Rightarrow |P'| \neq |P|$, a ponieważ $|P'| \geq |P|$, otrzymujemy $|P| < |P'|$. $\square$

Zadanie 2 — matroid podzbiorów krawędzi o ograniczonym stopniu (15 p.)

G spójny dwudzielny, $S \in \mathcal{F} \iff$ każdy wierzchołek incydentny z S ma stopień $\leq k$.
Czy $(E, \mathcal{F})$ zawsze jest matroidem?

Nie — kontrprzykład. Dla $k = 1$ rodzina $\mathcal{F}$ to rodzina skojarzeń w G . Weźmy ścieżkę P_4 na czterech wierzchołkach z krawędziami e_1, e_2, e_3 (kolejno); jest to graf spójny i dwudzielny. Rozważmy

$$I_1 = \{e_2\} \in \mathcal{F}, \quad I_2 = \{e_1, e_3\} \in \mathcal{F}, \quad |I_1| < |I_2|.$$

Aby zachować własność wymiany, powinniśmy móc dodać do I_1 pewną krawędź z $I_2 \setminus I_1$. Jednak e_1 i e_2 mają wspólny wierzchołek (stopień 2 > 1), tak samo e_2 i e_3 . Zatem ani $\{e_2, e_1\}$, ani $\{e_2, e_3\}$ nie należą do $\mathcal{F}$ — własność wymiany jest złamana. Para $(E, \mathcal{F})$ nie jest więc matroidem. $\square$

Zadanie 3 — mnożenie dużych liczb metodą Tooma-Cooka (15 p.)

Wykorzystując dane wzory (3 części, mnożenia x_2y_2 , x_0y_0 , A , B , C), zaprojektuj algorytm mnożenia dużych liczb i oszacuj jego czas.

Algorytm (Toom–Cook 3-way). Liczbę n -cyfrową zapisujemy w bazie 10^k , $k = \lceil n/3 \rceil$, jako $x_210^{2k} + x_110^k + x_0$ (i analogicznie dla drugiej liczby); każda część x_i, y_i ma $\approx n/3$ cyfr. Liczymy pięć iloczynów

$$x_2y_2, \quad x_0y_0, \quad A, \quad B, \quad C,$$

gdzie A, B, C to iloczyny sum/różnic części z małymi stałymi współczynnikami $(2, 4)$; mnożniki te mają nadal $\mathcal{O}(n/3)$ cyfr, więc to 5 mnożeń liczb rozmiaru $\approx n/3$ (rekurencyjnie). Z tych pięciu iloczynów składamy wynik danymi wzorami: pozostałe operacje to dodawania, odejmowania, przesunięcia (mnożenia przez 10^k) oraz dzielenia przez stałe 2 i 6 — wszystko w czasie $\mathcal{O}(n)$.

Złożoność.

$$T(n) = 5T\left(\frac{n}{3}\right) + \mathcal{O}(n).$$

Ponieważ $\log_3 5 \approx 1,465 > 1$, dominuje człon rekurencyjny:

$$T(n) = \Theta(n^{\log_3 5}) \approx \Theta(n^{1,465}),$$

czyli asymptotycznie szybciej niż Karacuba ($n^{\log_2 3} \approx n^{1,585}$). $\square$

Zadanie 4 — najkrótszy cykl Hamiltona (15 p.)

Algorytm znajdujący długość najkrótszego cyklu Hamiltona w grafie ważonym o n wierzchołkach w czasie $\mathcal{O}(2^n)$ (z dokładnością do wielomianu).

Programowanie dynamiczne Helda–Karpa. Ustalamy wierzchołek startowy 1. Dla $S \subseteq V$ z $1 \in S$ oraz $v \in S$ niech $D[S][v]$ będzie minimalną wagą ścieżki, która zaczyna się w 1, odwiedza dokładnie zbiór S i kończy się w v . Wtedy

$$D[\{1\}][1] = 0, \quad D[S][v] = \min_{u \in S \setminus \{v\}} \left(D[S \setminus \{v\}][u] + w(u, v) \right).$$

Długość najkrótszego cyklu Hamiltona to

$$\min_{v \neq 1} \left(D[V][v] + w(v, 1) \right).$$

Poprawność. $D[S][v]$ poprawnie liczy optymalną ścieżkę przez S kończącą się w v : przedostatni wierzchołek u jednoznacznie wyznacza prefiks odwiedzający $S \setminus \{v\}$ i kończący się w u , optymalizowany rekurencyjnie (podstruktura optymalna). Dodanie krawędzi powrotnej $w(v, 1)$ domyka cykl Hamiltona; minimalizacja po v daje optimum.

Złożoność. Stanów $D[S][v]$ jest $\mathcal{O}(2^n n)$, a każdy liczymy w czasie $\mathcal{O}(n)$:

$$\mathcal{O}(2^n n^2) = \mathcal{O}^*(2^n). \quad \square$$

Zadanie 5 — 3-aproksymacyjny zbiór niezależny w grafach dysków jednostkowych (15 p.)

Wielomianowy algorytm 3-aproksymacyjny dla najliczniejszego zbioru niezależnego w grafach dysków jednostkowych.

Korzystamy z tego samego faktu geometrycznego, co przy kolorowaniu (podrozdz. A.2): zbiór punktów z prawej półpłaszczyzny koła o promieniu 1 wokół najbardziej lewego punktu p rozkłada się na 3 wycinki 60° , z których każdy jest kliką.

Algorytm (zachłanne „obieranie” od lewej).

1. $\mathcal{S} \leftarrow \emptyset$.
2. Dopóki zostały punkty: wybierz najbardziej lewy (najmniejsze x) punkt p , dodaj p do $\mathcal{S}$ i usuń p oraz wszystkich jego sąsiadów $N(p)$.
3. Zwróć $\mathcal{S}$.

$\mathcal{S}$ jest niezależny: dodając p usuwamy wszystkich jego sąsiadów, więc żadne dwa wybrane punkty nie sąsiadują.

Współczynnik aproksymacji. Niech $p_1, p_2, \dots, p_t$ będą kolejno wybranymi punktami; usuwane domknięte sąsiedztwa $N[p_i] = \{p_i\} \cup N(p_i)$ są parami rozłączne i pokrywają wszystkie wierzchołki. Niech OPT będzie najliczniejszym zbiorem niezależnym. Dla każdego i punkty $N(p_i)$ leżą (w chwili wyboru, gdy p_i jest najbardziej lewy) w prawej półpłaszczyźnie koła, więc rozkładają się na 3 kliki. Zbiór niezależny zawiera co najwyżej jeden punkt z każdej kliki, więc

$$|\text{OPT} \cap N[p_i]| \leq 3.$$

(Jeśli $p_i \in \text{OPT}$, to żaden jego sąsiad nie jest w OPT , więc wkład wynosi 1.) Ponieważ sąsiedztwa $N[p_i]$ pokrywają wszystkie wierzchołki i są rozłączne,

$$|\text{OPT}| = \sum_{i=1}^t |\text{OPT} \cap N[p_i]| \leq 3t = 3|\mathcal{S}|,$$

czyli $|\mathcal{S}| \geq |\text{OPT}|/3$. Algorytm jest 3-aproksymacyjny i działa w czasie wielomianowym. $\square$

2021 — rozwiązania

Termin 1

Zadanie 1 — przecinające się kwadraty jednostkowe (15 p.)

Zaprojektuj rekurencyjny algorytm w oparciu o strategię „dziel i zwyciężaj” rozwiązujący następujący problem. *Dane*: zbiór kwadratów na płaszczyźnie o boku jeden, których boki są równoległe do osi. *Szukane*: para przecinających się kwadratów lub odpowiedź, że takiej pary nie ma. Oblicz złożoność obliczeniową zaprojektowanego algorytmu.

Sprowadzenie do pary najbliższych punktów. Każdy kwadrat o boku 1 i bokach równoległych do osi reprezentujemy jego środkiem $p_i = (x_i, y_i)$. Dwa takie kwadraty p_i, p_j przecinają się (mają wspólny punkt) wtedy i tylko wtedy, gdy nakładają się ich rzuty na obie osie:

$$|x_i - x_j| \leq 1 \quad \text{oraz} \quad |y_i - y_j| \leq 1, \quad \text{czyli} \quad d_\infty(p_i, p_j) = \max(|x_i - x_j|, |y_i - y_j|) \leq 1.$$

(Dla samych wnętrz zamieniamy $\leq$ na $<$; nie zmienia to algorytmu.) Szukamy więc pary punktów w odległości $d_\infty \leq 1$ albo stwierdzamy, że jej nie ma. Wystarczy znaleźć parę najbliższą w metryce d_∞ i porównać jej odległość z 1: jeśli minimalna odległość > 1 , to żadna para się nie przecina.

Algorytm. Stosujemy schemat „dziel i zwyciężaj” dla pary najbliższych punktów (notatki, wykład o parze najbliższych punktów: rys. 1), zmieniając jedynie metrykę z euklidesowej na d_∞ :

- **Dziel.** Sortujemy punkty po x (raz, wstępnie) i dzielimy pionową prostą l na dwie połowy po $\lfloor n/2 \rfloor$ punktów.
- **Zwyciężaj.** Rekurencyjnie wyznaczamy najmniejsze δ_L, δ_R w obu połowach; $\delta = \min(\delta_L, \delta_R)$.
- **Połącz.** W pasie $|x - l| \leq \delta$ (punkty posortowane po y) każdy punkt porównujemy tylko z następnymi, których współrzędna y różni się o $\leq \delta$. Jak w dowodzie dla pary najbliższych punktów, w prostokącie $2\delta \times \delta$ leży $\mathcal{O}(1)$ punktów (gdyby było ich więcej, dwa leżałyby w odległości $d_\infty < \delta$ po tej samej stronie — sprzeczność z δ), więc faza łączenia kosztuje $\mathcal{O}(n)$.

Dla $n \leq 3$ liczymy wszystkie pary wprost. Na końcu zwracamy najbliższą parę, gdy jej $d_\infty \leq 1$, w przeciwnym razie „brak pary”.

Złożoność. Wstępne sortowanie $\mathcal{O}(n \log n)$; rekurencja $T(n) = 2T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n)$. Łącznie $\boxed{\mathcal{O}(n \log n)}$.

Zadanie 2 — liczba etykietowań $L(2,0)$ ścieżki (15 p.)

Etykietowaniem $L(2,0)$ grafu G nazywamy funkcję przyporządkowującą wierzchołkom G liczby ze zbioru $\{1, 2, \dots, k\}$ taką, że sąsiednie wierzchołki otrzymują etykiety różniące się o co najmniej 2. Zaprojektuj wielomianowy algorytm programowania dynamicznego zliczający liczbę etykietowań $L(2,0)$ czterema etykietami ($k = 4$) dla ścieżki o n wierzchołkach. Oszacuj asymptotyczną złożoność (w terminach $\mathcal{O}(\cdot)$).

Etykietujemy wierzchołki ścieżki $v_1, \dots, v_n$ liczbami ze zbioru $\{1, 2, 3, 4\}$ tak, że sąsiednie etykiety różnią się o co najmniej 2. Dwie etykiety a, b są zgodne, gdy $|a - b| \geq 2$; zgodne pary (nieuporządkowane) to

$$\{1, 3\}, \{1, 4\}, \{2, 4\} \quad (\text{oraz } a = b \text{ jest zawsze niezgodne}).$$

Programowanie dynamiczne. Niech $D[i][c]$ to liczba poprawnych etykietowań prefiksu $v_1, \dots, v_i$, w których v_i ma etykietę c . Wtedy

$$D[1][c] = 1 \quad (c = 1, \dots, 4), \quad D[i][c] = \sum_{\substack{c'=1 \\ |c-c'| \geq 2}}^4 D[i-1][c'],$$

a odpowiedzią jest $\sum_{c=1}^4 D[n][c]$. Tablicę wypełniamy dla $i = 2, \dots, n$; każda komórka to suma po ≤ 4 poprzednikach.

Złożoność. n wierszy $\times 4$ etykiety $\times \mathcal{O}(1)$ pracy na komórkę: $\boxed{\mathcal{O}(n)}$ czasu i $\mathcal{O}(1)$ pamięci (trzymamy tylko wiersz $i - 1$).

Zadanie 3 — najmniejsza transversala (15 p.)

Niech $\mathcal{S}$ będzie rodziną pewnych podzbiorów zbioru U . *Transwersalą* dla rodziny $\mathcal{S}$ nazywamy dowolny podzbiór U przecinający każdy zbiór z $\mathcal{S}$. Podaj algorytm aproksymacyjny dla problemu znajdowania najmniejszej transversali przy założeniu, że każdy element zbioru U należy do co najwyżej dwóch zbiorów z $\mathcal{S}$. *Wskazówka:* sprowadź ten problem do problemu pokrycia zbioru.

Dana rodzina $\mathcal{S}$ podzbiorów zbioru U ; transwersala to $T \subseteq U$ przecinająca każdy zbiór z $\mathcal{S}$. Zakładamy, że każdy element U należy do co najwyżej dwóch zbiorów z $\mathcal{S}$.

Sprowadzenie do pokrycia zbioru. Tworzymy instancję pokrycia zbioru, w której pokrywamy rodzinę $\mathcal{S}$: zbiorem bazowym jest $X = \mathcal{S}$, a dla każdego elementu $u \in U$ definiujemy zbiór pokrywający

$$S_u = \{ A \in \mathcal{S} : u \in A \} \subseteq \mathcal{S}.$$

Wybranie elementu u do transversali „pokrywa” dokładnie te A , które u przecina. Zatem $\{u_1, \dots, u_t\}$ jest transversalą wtedy i tylko wtedy, gdy $S_{u_1}, \dots, S_{u_t}$ pokrywają $X = \mathcal{S}$, przy równym koszcie t . Najmniejsza transversala = najmniejsze pokrycie tej instancji.

Algorytm i współczynnik. Założenie „każdy u w co najwyżej dwóch zbiorach” daje $|S_u| \leq 2$ dla każdego u . Uruchamiamy zachłanny algorytm pokrycia zbioru (GREEDY-SET-COVER, Algorytm 17). Z twierdzenia o jego jakości jest on H_d -aproksymacyjny, gdzie $d = \max_u |S_u|$ to liczność największego zbioru pokrywającego. Tu $d \leq 2$, więc współczynnik wynosi

$$H_2 = 1 + \frac{1}{2} = \frac{3}{2},$$

a zatem otrzymujemy algorytm $\frac{3}{2}$ -aproksymacyjny (w szczególności 2-aproksymacyjny) dla najmniejszej transversali.

Zadanie 4 — ze schematu prawie-optimalnego do 2-aproksymacji (15 p.)

Niech P będzie problemem optymalizacyjnym, w którym poszukujemy rozwiązania dopuszczalnego maksymalizującego funkcję celu f , której zbiorem wartości jest przedział $[3, \infty)$. Wiadomo, że dla problemu P i każdego $\epsilon > 0$ istnieje algorytm wielomianowy, który dla wejścia I znajduje rozwiązanie dopuszczalne $A(I)$ takie, że $f(A(I)) \geq (1 - \epsilon)f(OPT(I)) - 1$, gdzie $OPT(I)$ to rozwiązanie optymalne. Udowodnij, że istnieje wielomianowy algorytm 2-aproksymacyjny dla problemu P .

Dla każdego $\epsilon > 0$ mamy wielomianowy A_ϵ z gwarancją $f(A_\epsilon(I)) \geq (1 - \epsilon)f(OPT(I)) - 1$, a wartości f leżą w $[3, \infty)$, więc $f(OPT(I)) \geq 3$ dla każdej instancji. Chcemy algorytmu 2-aproksymacyjnego, tj. $f(A(I)) \geq \frac{1}{2}f(OPT(I))$.

Konstrukcja. Ustalamy $\epsilon = \frac{1}{6}$ (stała, więc $A_{1/6}$ pozostaje wielomianowy). Oznaczmy $V = f(OPT(I)) \geq 3$. Wtedy

$$f(A_{1/6}(I)) \geq \left(1 - \frac{1}{6}\right)V - 1 = \frac{5}{6}V - 1 = \frac{1}{2}V + \left(\frac{1}{3}V - 1\right) \geq \frac{1}{2}V,$$

bo $\frac{1}{3}V - 1 \geq 0 \iff V \geq 3$, co zachodzi. Zatem $A_{1/6}$ jest wielomianowym algorytmem 2-aproksymacyjnym dla P . $\square$

Termin 2

Zadanie 1 — scalanie list (15 p.)

- (i) (3p.) Niech A, B będą listami długości k i l posortowanymi rosnąco. Podaj algorytm działający w czasie $\mathcal{O}(k + l)$, który scala je w jedną posortowaną rosnąco listę długości $k + l$.
- (ii) (4p.) Mamy m list długości n każda, wszystkie posortowane rosnąco; scalamy je w jedną długości mn , używając wielokrotnie algorytmu z (i): scalamy dwie pierwsze listy, wynik z trzecią, ten z czwartą, itd. Oszacuj złożoność jako funkcję m i n .
- (iii) (8p.) Podaj asymptotycznie szybszy algorytm typu „dziel i zwyciężaj” rozwiązujący problem z (ii). Oszacuj jego złożoność.

(i) **Scalanie dwóch list w czasie $\mathcal{O}(k + l)$.** Klasyczne scalanie dwoma wskaźnikami (procedura MERGE z sortowania przez scalanie; por. MERGELISTS z notatek o sumie podzbiorów, Algorytm 18):

Algorytm 4: Merge — scalanie dwóch list posortowanych

```

1: function MERGE( $A[1 \dots k], B[1 \dots l]$ )
2:    $i \leftarrow 1; j \leftarrow 1; C \leftarrow$  pusta lista
3:   while  $i \leq k$  and  $j \leq l$  do
4:     if  $A[i] \leq B[j]$  then
5:       dołącz  $A[i]$  do  $C$ ;  $i \leftarrow i + 1$ 
6:     else
7:       dołącz  $B[j]$  do  $C$ ;  $j \leftarrow j + 1$ 
8:   dołącz do  $C$  pozostałe elementy  $A$  lub  $B$ 
9:   return  $C$ 

```

Każda iteracja przesuwaa o jeden któryś ze wskaźników, więc czas $\mathcal{O}(k + l)$.

(ii) **Scalanie iteracyjne.** Scalamy listy po kolei: wynik długości n z kolejną listą długości n , potem długości $2n$ z następną, itd. Scalanie przed dołączeniem i -tej listy ma rozmiar $(i - 1)n$, więc kosztuje $\mathcal{O}((i - 1)n + n) = \mathcal{O}(in)$. Łącznie

$$\sum_{i=2}^m \mathcal{O}(in) = \mathcal{O}\left(n \sum_{i=2}^m i\right) = \boxed{\mathcal{O}(m^2 n)}.$$

(iii) **Scalanie „dziel i zwyciężaj”.** Scalamy listy parami (poziom 1: $m/2$ scaleń list długości n), potem powstałe pary parami, itd. — jak w sortowaniu przez scalanie po listach jako liściach. Na każdym z $\lceil \log_2 m \rceil$ poziomów łączna długość scalanych danych wynosi mn , więc poziom kosztuje $\mathcal{O}(mn)$. Stąd

$$T(m, n) = \boxed{\mathcal{O}(mn \log m)},$$

asymptotycznie szybciej niż $\mathcal{O}(m^2 n)$ z podpunktu (ii).

Zadanie 2 — brak słów kodowych długości 1 (15 p.)

Załóżmy, że w tekście, który chcemy zakodować za pomocą kodu Huffmana, każdy znak występuje mniej niż $\frac{24}{3}$ razy. Udowodnij, że w kodzie Huffmana dla takiego tekstu żaden znak nie będzie zakodowany za pomocą tylko jednego bitu.

Lemat B.1. *Jeśli w tekście o łącznej liczbie wystąpień T pewien znak x otrzymuje w kodzie Huffmana słowo długości 1, to $f(x) \geq T/3$.*

Dowód. Słowo długości 1 znaczy, że liść x jest dzieckiem korzenia. Drugim poddrzewem korzenia jest R obejmujące wszystkie pozostałe znaki, o wadze $f(R) = T - f(x)$. Przy ≥ 3 znakach R jest węzłem wewnętrznym, powstałym ze scalenia dwóch węzłów o wagach a, b z $a + b = T - f(x)$. W kroku, w którym powstał korzeń R , węzeł x był jeszcze dostępny, a mimo to scalono a i b — czyli były to dwa najlżejsze węzły, więc $f(x) \geq a$ oraz $f(x) \geq b$. Zatem

$$f(x) \geq \max(a, b) \geq \frac{a+b}{2} = \frac{T - f(x)}{2} \implies 3f(x) \geq T \implies f(x) \geq \frac{T}{3}.$$

□

Wniosek (teza zadania). Jeśli każdy znak występuje mniej niż $T/3$ razy (np. mniej niż $\frac{24}{3} = 8$ razy w tekście długości $T = 24$), to żaden znak nie spełnia $f(x) \geq T/3$, więc — na mocy lematu — żaden znak nie otrzyma słowa kodowego długości 1. □

Zadanie 3 — pakowanie do skrzyń (Next-Fit) jest 2-aproksymacyjne (15 p.)

Rozważmy następujący algorytm dla problemu pakowania do skrzyń. Wejściem jest nieposortowana lista rozmiarów pakowanych obiektów. Algorytm pakuje obiekty w kolejności z listy, próbując umieścić kolejny obiekt tylko w ostatnio rozpoczętej skrzyni; jeśli to niemożliwe, rozpoczyna nową skrzynię. Udowodnij, że jest to algorytm 2-aproksymacyjny dla problemu pakowania do skrzyń.

Skrzynie mają pojemność 1, rozmiary obiektów $s_1, \dots, s_n \in (0, 1]$. Algorytm (*Next-Fit*) pakuje obiekt do ostatnio rozpoczętej skrzyni, a gdy się nie mieści — rozpoczyna nową. Niech $S = \sum_i s_i$ i niech algorytm użyje k skrzyń, a optimum OPT .

Dolne ograniczenie optimum. Żadne upakowanie nie zmieści więcej niż 1 w skrzyni, więc $OPT \geq \lceil S \rceil \geq S$.

Górne ograniczenie dla Next-Fit. Rozważmy dwie kolejne skrzynie B_i, B_{i+1} . Pierwszy obiekt trafiający do B_{i+1} nie zmieścił się w B_i , więc suma zawartości obu skrzyń przekracza pojemność: $\text{cont}(B_i) + \text{cont}(B_{i+1}) > 1$ dla $i = 1, \dots, k-1$. Sumując te nierówności i korzystając z $\sum_i \text{cont}(B_i) = S$:

$$2S = \sum_{i=1}^k 2 \text{cont}(B_i) \geq \sum_{i=1}^{k-1} (\text{cont}(B_i) + \text{cont}(B_{i+1})) > k - 1.$$

Stąd $k < 2S + 1$, a ponieważ k jest całkowite, $k \leq 2S \leq 2OPT$.

Wniosek. $k \leq 2OPT$, więc Next-Fit jest algorytmem 2-aproksymacyjnym. $\square$

Zadanie 4 — najdłuższy wspólny podciąg kolejnych wyrazów (15 p.)

Dla zadanych ciągów $(x_1, x_2, \dots, x_n)$ i $(y_1, y_2, \dots, y_m)$ chcemy znaleźć długość ich najdłuższego wspólnego podciągu *kolejnych* wyrazów, czyli największe k , dla którego istnieją indeksy i, j takie, że $(x_i, x_{i+1}, \dots, x_{i+k-1}) = (y_j, y_{j+1}, \dots, y_{j+k-1})$. Podaj algorytm programowania dynamicznego o złożoności $\mathcal{O}(mn)$ rozwiązujący ten problem.

Szukamy najdłuższego wspólnego spójnego fragmentu (substring) ciągów $x_{1\dots n}$ i $y_{1\dots m}$. W odróżnieniu od LCS z wykładu fragment musi być ciągły.

Programowanie dynamiczne. Niech $D[i][j]$ to długość najdłuższego wspólnego *sufiksu* prefiksów $x_{1\dots i}$ i $y_{1\dots j}$ (czyli wspólnego fragmentu kończącego się dokładnie na x_i i y_j):

$$D[i][j] = \begin{cases} D[i-1][j-1] + 1, & x_i = y_j, \\ 0, & x_i \neq y_j, \end{cases} \quad D[0][j] = D[i][0] = 0.$$

Odpowiedzią jest $\max_{i,j} D[i][j]$ (indeksy początku odzyskujemy z pozycji maksimum: $i - k + 1, j - k + 1$).

Złożoność. $(n+1)(m+1)$ komórek, $\mathcal{O}(1)$ pracy każda: $\boxed{\mathcal{O}(mn)}$ czasu (pamięć $\mathcal{O}(m)$ przy trzymaniu dwóch wierszy).

Termin 3

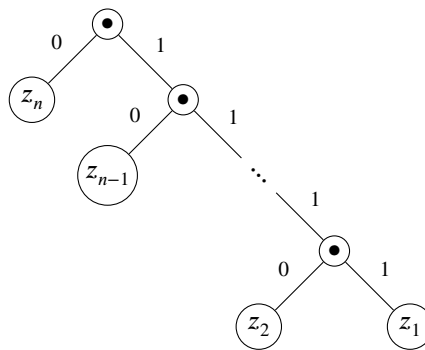
Zadanie 1 — kod Huffmana dla częstości będących potęgami dwójki (15 p.)

Alfabet A składa się ze znaków $z_1, z_2, \dots, z_n$. W pewnym tekście T znak z_i występuje 2^{i-1} razy dla wszystkich $i \in \{1, 2, \dots, n\}$. Znajdź kod Huffmana dla tekstu T .

Znak z_i występuje 2^{i-1} razy, $i = 1, \dots, n$ (częstości $1, 2, 4, \dots, 2^{n-1}$). Kluczowa obserwacja:

$$\sum_{j=1}^{i-1} 2^{j-1} = 2^{i-1} - 1 < 2^{i-1},$$

więc waga węzła scalającego pierwsze $i - 1$ znaków jest mniejsza od częstości kolejnego znaku z_i . Dlatego Huffman buduje „grzebień”: najpierw scala z_1 z z_2 , wynik z z_3 , ten z z_4 , itd.



Rysunek 2: Drzewo Huffmana dla częstości 2^{i-1} — „grzebień”.

Słowa kodowe i długości. Przyjmując konwencję „lewo = 0, prawo = 1” z rysunku:

$$z_n : 0, \quad z_{n-1} : 10, \quad z_{n-2} : 110, \quad \dots, \quad z_2 : \underbrace{1 \dots 1}_{n-2} 0, \quad z_1 : \underbrace{1 \dots 1}_{n-1}.$$

Długości: $\ell(z_i) = n - i + 1$ dla $i \geq 2$ oraz $\ell(z_1) = n - 1$ (oba najgłębsze znaki z_1, z_2 na poziomie $n - 1$). Sprawdzenie Krafta: $2 \cdot 2^{-(n-1)} + \sum_{i=3}^n 2^{-(n-i+1)} = 2^{-(n-2)} + (1 - 2^{-(n-2)}) = 1$.

Zadanie 2 — rozdział czekolady (15 p.)

Kostka czekolady o wadze $2n$ dkg spadła na podłogę i połamano się na wiele kawałków, z których żaden nie waży więcej niż 1 dkg. Zaprojektuj algorytm o liniowej złożoności obliczeniowej, który rozdziela te kawałki pomiędzy n dzieci tak, że każde dziecko dostaje kawałki o sumarycznej wadze co najmniej 1 dkg. Udowodnij, że Twój algorytm jest poprawny.

Kostka waży $2n$ dkg, połamana na kawałki, każdy ≤ 1 dkg; rozdzielamy je n dzieciom tak, by każde dostało ≥ 1 dkg.

Algorytm (liniowy). Przeglądamy kawałki w dowolnej ustalonej kolejności. Dla dziecka $j = 1, \dots, n-1$ dokładamy kolejne kawałki, dopóki jego suma nie osiągnie ≥ 1 dkg, po czym przechodzimy do dziecka $j+1$. Wszystkie pozostałe kawałki oddajemy dziecku n . Każdy kawałek dotykamy raz: czas $\mathcal{O}(\#\text{kawałków})$, liniowy.

Poprawność. Gdy dziecko $j < n$ kończymy obsługiwać, jego suma t_j spełnia $1 \leq t_j < 2$: przed ostatnim kawałkiem było < 1 , a kawałek dokłada ≤ 1 . Stąd pierwszych $n-1$ dzieci zużywa $\sum_{j < n} t_j < 2(n-1)$ dkg, więc dla dziecka n zostaje

$$R = 2n - \sum_{j < n} t_j > 2n - 2(n-1) = 2 \geq 1 \text{ dkg.}$$

Zatem każde z n dzieci dostaje ≥ 1 dkg — algorytm nie „zacina się” przed obsłużeniem wszystkich. $\square$

Zadanie 3 — stacje przekaznikowe (DP) (15 p.)

Firma „Głuchy Telefon” buduje stacje przekaznikowe wzdłuż autostrady A44 o długości M km. Jest n możliwych lokalizacji, leżących w odległościach $m_1 < m_2 < \dots < m_n$ od początku autostrady (wszystkie przy autostradzie). Koszt budowy stacji w lokalizacji i -tej to $k_i > 0$, $i \in \{1, \dots, n\}$. Stacje trzeba wybudować tak, by z każdego punktu autostrady było nie dalej niż k km do najbliższej stacji. Podaj efektywny algorytm programowania dynamicznego obliczający najmniejszy całkowity koszt inwestycji i oszacuj jego złożoność obliczeniową.

Autostrada $[0, M]$, możliwe lokalizacje w $m_1 < \dots < m_n$, koszty $k_i > 0$. Stacja w m_i obsługuje punkty w zasięgu k , czyli przedział $[m_i - k, m_i + k]$. Wybieramy podzbiór lokalizacji o minimalnym koszcie, którego przedziały pokrywają całe $[0, M]$.

Programowanie dynamiczne. Niech $D[i]$ to minimalny koszt zbioru stacji, w którym stacja i jest najdalej wysuniętą wybraną stacją, a odcinek $[0, m_i + k]$ jest w pełni pokryty (bez luk). Wtedy

$$D[i] = k_i + \begin{cases} 0, & m_i - k \leq 0 \quad (\text{stacja } i \text{ pokrywa początek}), \\ \min_{\substack{j < i \\ m_j + k \geq m_i - k}} D[j], & \text{w przeciwnym razie,} \end{cases}$$

gdzie warunek $m_j + k \geq m_i - k$ (tj. $m_i - m_j \leq 2k$) gwarantuje, że zasięg poprzedniej stacji j styka się z zasięgiem i — nie powstaje luka. Gdy oba przypadki są możliwe, bierzemy mniejszy; gdy żaden — $D[i] = \infty$. Odpowiedzią jest

$$\min_{i: m_i + k \geq M} D[i]$$

(jeśli zbiór jest pusty, pokrycie jest niewykonalne).

Złożoność. Dla każdego i minimalizujemy po $j < i$: $\mathcal{O}(n^2)$. Ponieważ lokalizacje są posortowane, a warunek $m_i - m_j \leq 2k$ jest monotoniczny, można utrzymywać minimum przesuwającym oknem i zejść do $\boxed{\mathcal{O}(n)}$ (po $\mathcal{O}(n \log n)$ na sortowanie, jeśli dane nieposortowane).

Zadanie 4 — obiekt majoryzujący (dziel i zwyciężaj) (15 p.)

Tablica A zawiera n obiektów, które nie są liczbami (np. małe pliki tekstowe). Nie jest dla nich zdefiniowana relacja $>$, ale zdefiniowana jest równość (sprawdzana w czasie stałym), oznaczająca identyczność obiektów. Mówimy, że tablica A ma *obiekt majoryzujący* M , jeśli więcej niż połowa obiektów jest równa M . Podaj algorytm typu „dziel i zwyciężaj” o złożoności $\mathcal{O}(n \log n)$, który rozstrzyga, czy A zawiera obiekt majoryzujący, i jeśli tak — zwraca go.

Tablica $A[1 \dots n]$ obiektów z samą relacją równości (sprawdzaną w $\mathcal{O}(1)$); M majoryzuje, gdy występuje $> n/2$ razy.

Algorytm.

- **Baza:** dla $n = 1$ zwróć ten jeden obiekt.
- **Dziel:** podziel A na połowy L, R i rekurencyjnie wyznacz ich kandydatów na majoryzujący: c_L, c_R (lub „brak”).
- **Połącz:** jeśli obiekt majoryzuje całe A , to majoryzuje co najmniej jedną połowę (gdyby w obu występował $\leq$ połowy, łącznie miałyby $\leq n/2$). Wystarczy więc sprawdzić c_L i c_R : przeglądając A raz, zlicz wystąpienia każdego z nich i zwróć tego, który występuje $> n/2$ razy (jeśli żaden — „brak”).

Złożoność. $T(n) = 2T(n/2) + \mathcal{O}(n) = \boxed{\mathcal{O}(n \log n)}$.

Uwaga. Ten sam problem (przy samej równości) rozwiązuje liniowo głosowanie Boyera–Moore’a, ale zadanie wymaga schematu „dziel i zwyciężaj”.

Zadania różne

Zadanie 1 — liczba inwersji

Dla permutacji (ogólniej: tablicy) $A[1..n]$ policz liczbę par $i < j$ z $A[i] > A[j]$. Wiele przypadków testowych, $n < 1000$.

Zmodyfikowany sort przez scalanie. Liczbę inwersji liczymy „przy okazji” scalania. Sortowanie przez scalanie dzieli A na połowy, rekurencyjnie je sortuje i scala dwie posortowane połowy w jedną posortowaną listę (zob. Merge). Modyfikacja: scalanie zwraca też liczbę inwersji między-połówkowych. Gdy bieżący element prawej połowy $R[j]$ jest mniejszy od $L[i]$, to — ponieważ lewa połowa jest posortowana — $R[j]$ jest mniejszy od całego pozostałego sufiksu $L[i \dots k]$, czyli tworzy $k - i + 1$ inwersji naraz.

Algorytm 5: Count — scalanie zliczające inwersje

```

1: function COUNTANDMERGE( $L[1 \dots k]$ ,  $R[1 \dots l]$ )                                ▷  $L, R$  posortowane
2:    $i \leftarrow 1$ ;  $j \leftarrow 1$ ;  $C \leftarrow$  pusta lista;  $inv \leftarrow 0$ 
3:   while  $i \leq k$  and  $j \leq l$  do
4:     if  $L[i] \leq R[j]$  then                                                    ▷ brak inwersji
5:       dołącz  $L[i]$  do  $C$ ;  $i \leftarrow i + 1$ 
6:     else                                                                    ▷  $R[j] < L[i]$ , więc  $R[j]$  wyprzedza  $L[i \dots k]$ 
7:       dołącz  $R[j]$  do  $C$ ;  $j \leftarrow j + 1$ 
8:        $inv \leftarrow inv + (k - i + 1)$ 
9:   dołącz do  $C$  pozostałe elementy  $L$  lub  $R$ 
10:  return ( $C$ ,  $inv$ )

```

Funkcja rekurencyjna sumuje inv z obu wywołań i ze scalenia.

Poprawność. Każda inwersja (i, j) ma elementy albo w tej samej połowie (policzona rekurencyjnie), albo w różnych (policzona przy scalaniu, bo $A[i]$ trafia do lewej, $A[j]$ do prawej połowy). Podziały są rozłączne, więc każda inwersja liczona dokładnie raz.

Złożoność. $T(n) = 2T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n)$ na test; wartości do 10^{12} trzymamy w 64-bitowych liczbach (jak i wynik, który może sięgać $\binom{n}{2}$). □

Zadanie 2 — maksymalne skojarzenie w drzewie bez ścieżek powiększających

Znajdź maksymalne skojarzenie w drzewie. Zaprezentuj działanie, udowodnij poprawność, podaj złożoność.

Zachłannie od liści (DFS post-order). Ukorzeniamy drzewo w dowolnym wierzchołku i uruchamiamy DFS. Decyzję o skojarzeniu wierzchołka v podejmujemy dopiero po przetworzeniu wszystkich jego poddrzew: jeśli wówczas v jest jeszcze wolny i ma jakieś wolne dziecko c (takie, którego własne poddrzewo nie zdołało go skojarzyć), to dodajemy krawędź vc do M i oznaczamy oboje jako skojarzonych. Każdy stan „wolny/skojarzony” przekazujemy w górę: v wraca do swojego rodzica jako skojarzony albo wolny, co steruje decyzją rodzica.

Algorytm 6: Maksymalne skojarzenie w drzewie

```

1: function MATCH( $v$ )                                ▷ zwraca (skojarzenie poddrzewa, czy  $v$  skojarzony)
2:    $M \leftarrow \emptyset$ ;  $matched \leftarrow \text{false}$ 
3:   for all dziecko  $c$  wierzchołka  $v$  do
4:      $(M_c, matched_c) \leftarrow \text{MATCH}(c)$            ▷ najpierw całe poddrzewo  $c$ 
5:      $M \leftarrow M \cup M_c$ 
6:     if not  $matched$  and not  $matched_c$  then
7:        $M \leftarrow M \cup \{vc\}$ ;  $matched \leftarrow \text{true}$ 
8:   return ( $M, matched$ )

9:  $(M, \_) \leftarrow \text{MATCH}(\text{root})$ ; return  $M$ 

```

Liść nie ma dzieci, więc pętla się nie wykonuje i liść wraca *wolny*; jego rodzic, widząc wolne dziecko, zagarnia je (o ile sam jest jeszcze wolny). To realizuje regułę „skojarz liść z rodzicem” z dowodu poprawności.

Skojarzenie v z pierwszym wolnym dzieckiem *nie* przerywa pętli po dzieciach: pozostałe dzieci wciąż są odwiedzane (ich poddrzewa trzeba rozwiązać), tylko że v , będąc już zajęty, nie kojarzy się z żadnym z nich. Dopiero gdy wszystkie dzieci v są przetworzone, v wraca w górę do rodzica. Prześledzenie działania na konkretnym drzewie pozostawiamy Czytelnikowi.

Poprawność. Kluczowy fakt: istnieje maksymalne skojarzenie, w którym pewien liść jest skojarzony ze swoim rodzicem. Niech ℓ będzie liściem o rodzicu u i niech M^* będzie maksymalne. Jeśli $\ell \in M^*$, to skojarzone jako ℓu (jedyne sąsiady). Jeśli ℓ wolny, to u musi być skojarzony (inaczej $M^* \cup \{\ell u\}$ byłoby większe) z pewnym w ; wtedy $M^* - uw + \ell u$ jest skojarzeniem tej samej liczności, w którym $\ell u \in M$. Zatem wybór ℓu jest bezpieczny: skoro pewne najliczniejsze skojarzenie go zawiera, dołożenie ℓu nie pogarsza optimum. Po dołożeniu ℓu żadna inna krawędź skojarzenia nie może dotykać ℓ ani u , więc usuwamy oba wierzchołki (wraz z incydentnymi krawędziami); pozostaje mniejszy las, a najliczniejsze skojarzenie całego drzewa to $\{\ell u\}$ wraz z najliczniejszym skojarzeniem reszty. Postępujemy tak rekurencyjnie. Przebieg post-order *jest* tą rekurencją: zanim odwiedzimy dany wierzchołek, jego dzieci (a wcześniej ich poddrzewa) są już przetworzone i usunięte. Przez indukcję M jest skojarzeniem o maksymalnej liczności.

Złożoność. Jeden przebieg post-order, $\mathcal{O}(1)$ na wierzchołek: $\mathcal{O}(n)$ czasu i pamięci. $\square$

Zadanie 3 — FZI_STEF Stefan (maksymalny spójny zysk)

Dany ciąg n zysków/strat (możliwe ujemne), $1 \leq n \leq 10^5$. Wybierz spójny podciąg (być może pusty) o maksymalnej sumie.

Algorytm Kadane’a. Przesuwamy się od lewej, utrzymując cur — najlepszą sumę spójnego podciągu *kończącego się* w bieżącym mieście — oraz globalne maksimum $best$. Aktualizacja $cur \leftarrow \max(0, cur + a_i)$: zero odpowiada „rozpocznij trasę później / zakończ wcześniej” (pusty wybór dozwolony, stąd wynik ≥ 0).

Algorytm 7: Kadane — maksymalna suma spójnego podciągu

```

1: function MAXPROFIT( $a[1 \dots n]$ )
2:    $cur \leftarrow 0$ ;  $best \leftarrow 0$ 
3:   for  $i \leftarrow 1$  to  $n$  do
4:      $cur \leftarrow \max(0, cur + a[i])$ 
5:      $best \leftarrow \max(best, cur)$ 
6:   return  $best$ 

```

Poprawność. cur po i -tym kroku to maksimum z 0 i najlepszego sufiksu kończącego się w i : albo dokładamy a_i do najlepszego podciągu kończącego się w $i-1$, albo zaczynamy na nowo (pusto). Każdy spójny podciąg kończy się w którymś i , więc $best$ przegląda wszystkie kandydaty. Sprawdzenie na przykładach $[1, -2, 4, 5, -2] \rightarrow 9$ oraz $[-1, -2] \rightarrow 0$ pozostawiamy Czytelnikowi.

Złożoność. Jeden przebieg, $\mathcal{O}(n)$ czasu, $\mathcal{O}(1)$ pamięci. $\square$

Zadanie 4 — liczba wszystkich pokryć wierzchołkowych drzewa

Policz, na ile sposobów można wybrać pokrycie wierzchołkowe drzewa (zbiór wierzchołków taki, że każda krawędź ma co najmniej jeden koniec w tym zbiorze). Liczymy wszystkie, nie tylko minimalne.

Programowanie dynamiczne na drzewie. Wybieramy dowolny korzeń r i ukorzeniamy drzewo (orientujemy krawędzie „od r w dół”), co nadaje każdemu wierzchołkowi rodzica oraz dzieci; przez T_v oznaczamy poddrzewo zakorzenione w v (wierzchołek v wraz ze wszystkim poniżej). Wybór korzenia jest dowolny — liczba pokryć całego drzewa od niego nie zależy. Dla dowolnego wierzchołka v definiujemy dwie liczby pokryć poddrzewa T_v :

- $f(v, 1)$ — liczba pokryć T_v , w których v jest w pokryciu;
- $f(v, 0)$ — liczba pokryć T_v , w których v nie jest.

Krawędź v –dziecko musi być pokryta. Jeśli $v \notin$ pokrycie, to każde dziecko *musi* być wybrane; jeśli $v \in$ pokrycie, dziecko jest dowolne:

$$f(v, 0) = \prod_{c \in \text{children}(v)} f(c, 1), \quad f(v, 1) = \prod_{c \in \text{children}(v)} (f(c, 0) + f(c, 1)).$$

Liść: poddrzewo T_v to sam wierzchołek v , bez krawędzi do pokrycia, więc każdy podzbiór $\{v\}$ jest pokryciem. Gdy v jest poza pokryciem, jedynym pokryciem jest zbiór pusty ($f(v, 0) = 1$); gdy v należy do pokrycia, jedynym jest $\{v\}$ ($f(v, 1) = 1$) — razem dwa pokrycia liścia. (Zgodnie ze wzorami: pusty iloczyn po braku dzieci równa się 1.)

Więc liczba wszystkich pokryć wierzchołkowych drzewa to $f(r, 0) + f(r, 1)$.

Algorytm 8: Liczba pokryć wierzchołkowych drzewa

```

1: function COUNT( $v$ )                                ▷ zwraca parę  $(f(v, 0), f(v, 1))$  przez DFS
2:    $f_0 \leftarrow 1$ ;  $f_1 \leftarrow 1$ 
3:   for all  $c \in \text{children}(v)$  do
4:      $(g_0, g_1) \leftarrow \text{COUNT}(c)$                 ▷ najpierw całe poddrzewo  $c$ 
5:      $f_0 \leftarrow f_0 \cdot g_1$ 
6:      $f_1 \leftarrow f_1 \cdot (g_0 + g_1)$ 
7:   return  $(f_0, f_1)$ 

8:  $(f_0, f_1) \leftarrow \text{COUNT}(\text{root})$ ; return  $f_0 + f_1$ 

```

Poprawność. Wybory w rozłącznych poddrzewach dzieci są niezależne (stąd iloczyny), a jedyny ograniczenie międzypoddrzewowe to pokrycie krawędzi vc — uwzględnione przez warunkowanie na przynależności v . Indukcja po wysokości daje poprawność.

Złożoność. $\mathcal{O}(1)$ pracy na krawędź, $\mathcal{O}(n)$ łącznie (liczby trzymamy modulo, jeśli zadana reszta, lub w wielkich liczbach). $\square$

Zadanie 5 — stonoga: najcięższy zbiór niezależny

Stonoga to drzewo, które po usunięciu liści staje się ścieżką („łodygą”). Dana jest ważona stonoga T o n wierzchołkach z wagami $w: V \rightarrow \mathbb{R}$. Znajdź zbiór niezależny o maksymalnej wadze metodą „dziel i zwyciężaj”.

Dygresja. Zbiór niezależny grafu to zbiór wierzchołków S , w którym żadne dwa wierzchołki nie sąsiadują — czyli żadna krawędź nie ma obu końców w S (to niezależność *grafowa*, nie *matroidowa*). Jest dopełnieniem pokrycia wierzchołkowego: S jest niezależny $\iff V \setminus S$ jest pokryciem wierzchołkowym. Szukamy S maksymalizującego $\sum_{v \in S} w(v)$.

Zwinięcie liści. Niech łodyga to ścieżka $s_1, \dots, s_m$; każdy s_i ma zbiór liści-nózek L_i (liście sąsiadują tylko ze swoim s_i). Liście jednego s_i są parami niezależne i sąsiadują tylko z s_i . Stąd dla każdego s_i mamy dwa warianty:

- *bierzemy s_i* : wkład $w(s_i)$; nóżek nie bierzemy (sąsiadują z s_i);
- *nie bierzemy s_i* : wkład $\ell_i := \sum_{x \in L_i} \max(0, w(x))$ (bierzemy wszystkie nóżki o dodatniej wadze, niezależne od całej reszty).

Problem redukuje się do najcięższego zbioru niezależnego na ścieżce, gdzie s_i ma dwie „wartości stanu”.

Dziel i zwyciężaj na ścieżce. Ścieżkę $s_1 \dots s_m$ dzielimy na środku. Rekurencja zwraca tablicę $M[a][d]$ ($a, d \in \{0, 1\}$): $M[a][d]$ to maksymalna waga niezależnego podzbioru danego fragmentu przy ustalonym stanie lewego krańca a i prawego d , gdzie 1 oznacza „kraniec wzięty”, a 0 „niewzięty”. Scalając dwa sąsiadujące fragmenty łączymy wyniki po wewnętrznych krańcach, zakazując „wzięty–wzięty” na styku. Dla lewej połówki o tablicy L i prawej o tablicy R wynik scalenia M to:

$$M[a][d] = \max_{\substack{b, c \\ \neg(b \wedge c)}} (L[a][b] + R[c][d]),$$

gdzie b to stan prawego krańca lewej połówki, a c — lewego krańca prawej (czyli stany wierzchołków sąsiadujących na styku). Pojedynczy s_i daje bazę: stan „wzięty” = $w(s_i)$, „niewzięty” = ℓ_i .

Algorytm 9: Najcięższy zbiór niezależny w stonodze

```

1: function SOLVE( $i, j$ )
2:   if  $i = j$  then
3:      $M[0][0] \leftarrow \ell_i$ ;  $M[1][1] \leftarrow w(s_i)$ 
4:      $M[0][1] \leftarrow M[1][0] \leftarrow -\infty$ 
5:     return  $M$ 
6:    $k \leftarrow \lfloor (i + j)/2 \rfloor$ 
7:    $L \leftarrow \text{SOLVE}(i, k)$ ;  $R \leftarrow \text{SOLVE}(k+1, j)$ 
8:   for all  $a, d \in \{0, 1\}$  do
9:      $M[a][d] \leftarrow \max_{\substack{b, c \in \{0, 1\} \\ \neg(b \wedge c)}} (L[a][b] + R[c][d])$ 
10:  return  $M$ 

11:  $\ell_i \leftarrow \sum_{x \in L_i} \max(0, w(x))$  dla każdego  $s_i$ 
12:  $M \leftarrow \text{SOLVE}(1, m)$ ; return  $\max_{a, d} M[a][d]$ 

```

$\triangleright$ zwraca tablicę $M[a][d]$ dla łodygi $s_i \dots s_j$
 $\triangleright$ baza: jeden wierzchołek, lewy kraniec = prawy
 $\triangleright$ scalenie po krawędzi $s_k s_{k+1}$
 $\triangleright$ zwinięcie liści

Poprawność i złożoność. Tablica brzegowa 2×2 jest pełnym „interfejsem” fragmentu: jedyne zależności między fragmentami to krawędź na styku, więc znajomość stanów krańców wystarcza

do poprawnego scalenia (własność optymalnej podstruktury oraz rozłączność podproblemów).

Zwinięcie liści (obliczenie wszystkich ℓ_i) odwiedza każdy wierzchołek raz przed rekurencją, czyli $\mathcal{O}(n)$.

Koszt $\text{SOLVE}(i, j)$ poza wywołaniami rekurencyjnymi: baza ($i = j$) to $\mathcal{O}(1)$;

podział k to $\mathcal{O}(1)$;

pętla scalająca przebiega 4 pary (a, d) , a każda to maksimum po stałej liczbie sum, więc $\mathcal{O}(1)$.

Dwa wywołania na połówkach o rozmiarze $m/2$ (gdzie m to długość łodygi; $m \leq n$) dają:

$$T(m) = 2 \cdot T\left(\frac{m}{2}\right) + \Theta(m^0) \qquad a = 2, \ b = 2, \ k = 0$$

$$2 > 2^0 \Rightarrow T(m) = \Theta(m^{\log_2 2}) = \Theta(m)$$

Razem z $\mathcal{O}(n)$ na zwinięcie liści i $m \leq n$ daje to łącznie $\mathcal{O}(n)$. $\square$

Zadanie 6 — lider (element większościowy) przez dziel i zwyciężaj

Lider to wartość występująca $> \lfloor n/2 \rfloor$ razy (jeśli istnieje, jest jedna). Znajdź go metodą „dziel i zwyciężaj”, bez sortowania, używając tylko porównań $==$.

Algorytm. Dla tablicy $A[1..n]$ niech $\text{LEADER}(A, p, r)$ zwraca lidera podtablicy $A[p..r]$ albo NIL, gdy go nie ma. Dzielimy $A[p..r]$ na środku ($q = \lfloor (p + r)/2 \rfloor$) na dwie rozłączne, pełne połowy $A[p..q]$ i $A[q+1..r]$ i rekurencyjnie znajdujemy lidera każdej z nich. Kluczowa obserwacja: jeśli x jest liderem całości, to jest liderem co najmniej jednej z połówek. Zatem jedynymi kandydatami są zwrócone wartości x_L i x_R ; dla każdego kandydata $\neq \text{NIL}$ zliczamy jego wystąpienia w *całej* podtablicy (jednym przebiegiem, porównania $==$) i zwracamy ten, którego licznik przekroczy $\lfloor (r - p + 1)/2 \rfloor$, inaczej NIL.

Algorytm 10: Lider przez dziel i zwyciężaj

```

1: function LEADER( $A, p, r$ )                                ▷ lider  $A[p..r]$  albo NIL
2:   if  $p = r$  then
3:     return  $A[p]$ 
4:    $q \leftarrow \lfloor (p + r)/2 \rfloor$ 
5:    $x_L \leftarrow \text{LEADER}(A, p, q)$ ;  $x_R \leftarrow \text{LEADER}(A, q+1, r)$ 
6:   for all  $x \in \{x_L, x_R\} \setminus \{\text{NIL}\}$  do                ▷ sprawdź kandydatów
7:      $c \leftarrow |\{i \in [p, r] : A[i] = x\}|$                 ▷ jeden przebieg, porównania  $==$ 
8:     if  $c > \lfloor (r - p + 1)/2 \rfloor$  then
9:       return  $x$ 
10:  return NIL

```

Dlaczego kandydat jest w połowce. Gdyby x nie był liderem żadnej połowy, to w $A[p..q]$ występuje $\leq \lfloor |A[p..q]|/2 \rfloor$ razy i podobnie w $A[q+1..r]$, więc łącznie $\leq \lfloor |A[p..q]|/2 \rfloor + \lfloor |A[q+1..r]|/2 \rfloor \leq \lfloor (r - p + 1)/2 \rfloor$ — sprzeczność z byciem liderem całości. Poprawność reszty: zliczenie w całej podtablicy rozstrzyga kandydata bezbłędnie.

Złożoność. Faza scalania to dwa zliczenia $\mathcal{O}(n)$, więc:

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + \Theta(n^1) \qquad a = 2, \ b = 2, \ k = 1$$

$$2 = 2^1 \Rightarrow T(n) = \Theta(n^1 \log n) = \Theta(n \log n)$$

□